

Verifying Rust Code with Verus

Chris Hawblitzel (and the Verus Team)



<https://github.com/verus-lang/verus/>

Contributors 84



[+ 70 contributors](#)

Verus: Floyd-Hoare Logic for Rust

Verus is:

- ... an implementation of **Floyd-Hoare logic**
- ... for the **Rust** programming Language
- ... using **SMT solvers** to automate proofs
- ... adding some higher-order logic, extensional =, induction, etc.

Inspired heavily by Dafny and F*

Similar Rust verification tools: Kani, Creusot, Prusti, Aeneas, ...

Verus demo

```
C:\verus\source> verus.exe binsearch.rs  
verification results:: 2 verified, 0 errors
```

```
fn binary_search(v: &Vec<u64>, key: u64) -> (ret: usize)  
  requires  
    forall |i: int, j: int| 0 <= i <= j < v.len() ==> v[i] <= v[j],  
    exists |i: int| 0 <= i < v.len() && key == v[i],  
  ensures  
    ret < v.len(),  
    key == v[ret as int],  
{  
  let mut left: usize = 0;  
  let mut right: usize = v.len() - 1;  
  while left != right  
    invariant  
      right < v.len(),  
      exists |i: int| left <= i <= right && key == v[i],  
      forall |i: int, j: int| 0 <= i <= j < v.len() ==> v[i] <= v[j],  
    decreases right - left  
  {  
    let mid = left + (right - left) / 2;  
    if v[mid] < key {  
      left = mid + 1;  
    } else {  
      right = mid;  
    }  
  }  
  left  
}
```

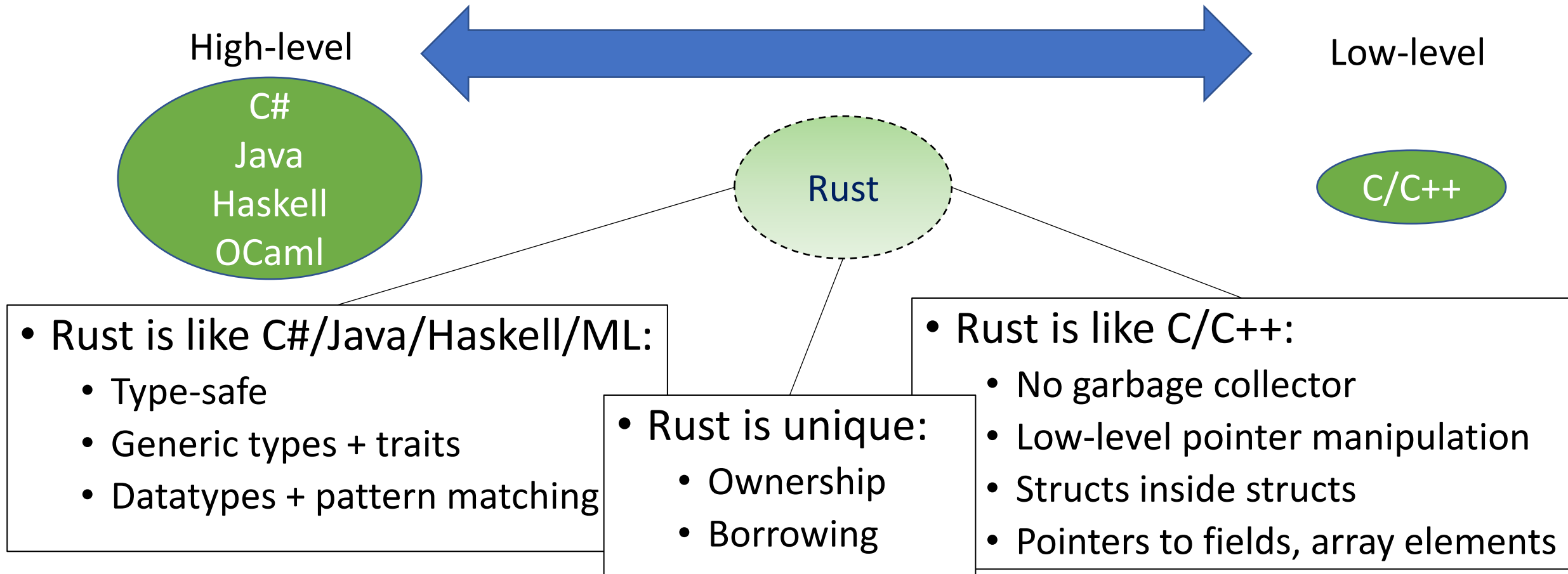
Verus demo

```
C:\verus\source> verus.exe binsearch.rs
error: invariant not satisfied at end of loop body
--> .\scratch.rs:29:13
29 |     exists|i: int| left <= i <= right && key == v[i],
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
error: aborting due to previous error

verification results:: 1 verified, 1 errors
```

```
fn binary_search(v: &Vec<u64>, key: u64) -> (ret: usize)
  requires
    forall|i: int, j: int| 0 <= i <= j < v.len() ==> v[i] <= v[j],
    exists|i: int| 0 <= i < v.len() && key == v[i],
  ensures
    ret < v.len(),
    key == v[ret as int],
{
  let mut left: usize = 0;
  let mut right: usize = v.len() - 1;
  while left != right
    invariant
      right < v.len(),
      exists|i: int| left <= i <= right && key == v[i],
      forall|i: int, j: int| 0 <= i <= j < v.len() ==> v[i] <= v[j],
      decreases right - left
    {
      let mid = left + (right - left) / 2;
      if v[mid] < key {
        left = mid + 2;
      } else {
        right = mid;
      }
    }
  left
}
```

Why write systems code in Rust?



Why write systems code in Rust?

“Speaking of languages, it's time to halt starting any new projects in C/C++ and use Rust for those scenarios where a non-GC language is required. For the sake of security and reliability, the industry should declare those languages as deprecated.”

- Mark Russinovich (Microsoft Azure CTO), 2022

Goal: verification for systems programmers

- No separate proof language or proof assistant
- **specs** + **proofs** + executable code in Rust

```
spec fn triangle(n: int) -> int
  decreases n
{
  if n <= 0 {
    0
  } else {
    n + triangle(n - 1)
  }
}
```

```
proof fn triangle_is_monotonic(i: int, j: int)
  ensures
    i <= j ==> triangle(i) <= triangle(j),
  decreases j
{
  if j > 0 {
    triangle_is_monotonic(i, j - 1);
  }
}
```

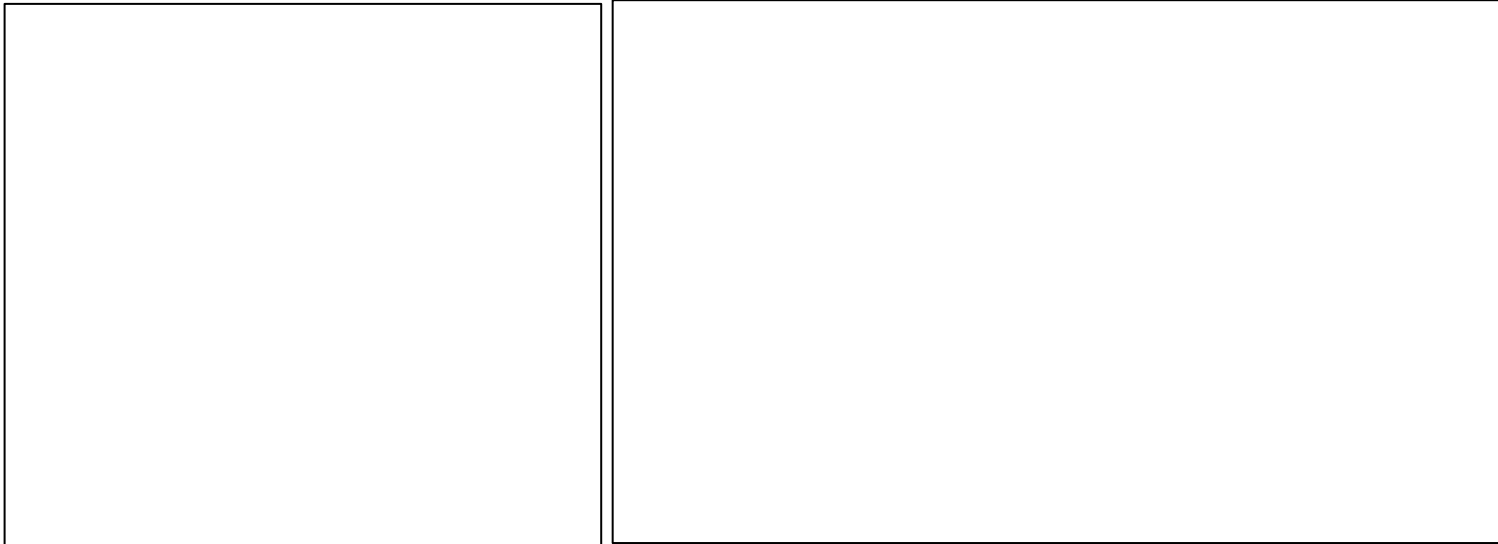
```
fn loop_triangle(n: u32) -> (sum: u32)
  requires triangle(n as int) <= u32::MAX
  ensures sum == triangle(n as int)
{
  let mut sum: u32 = 0;
  for idx in 0..n
    invariant
      sum == triangle(idx as int),
      triangle(n as int) <= u32::MAX,
    {
      assert(sum + (idx + 1) <= u32::MAX) by {
        triangle_is_monotonic(idx + 1, n as int);
      }
      sum = sum + (idx + 1);
    }
  sum
}
```

Note: spec fn, proof fn must terminate

- disallowed: spec fn f() -> bool { !f() }
- disallowed: proof fn f() ensures false { f(); }

Goal: verification for systems programmers

- Spec and proof code are “ghost”
 - ghost code erased via macro before compiling
 - can use regular rustc compiler



```
fn loop_triangle(n: u32) -> u32

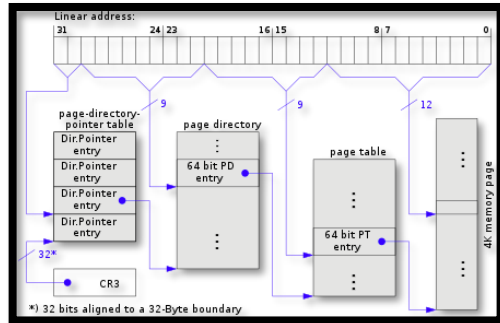
{
  let mut sum: u32 = 0;
  for idx in 0..n

  {

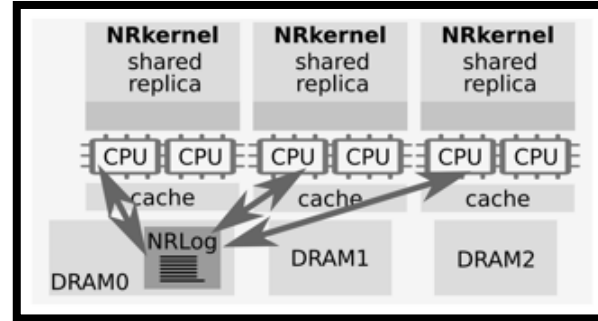
    sum = sum + (idx + 1);
  }
  sum
}
```

Example systems projects ($\geq 250K$ total public LoC)

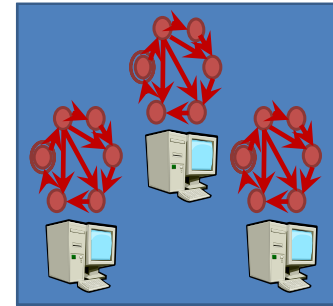
OS Page Table



NUMA-aware
Replica Library



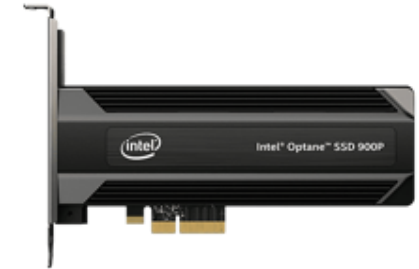
Distributed
Key-Value Store



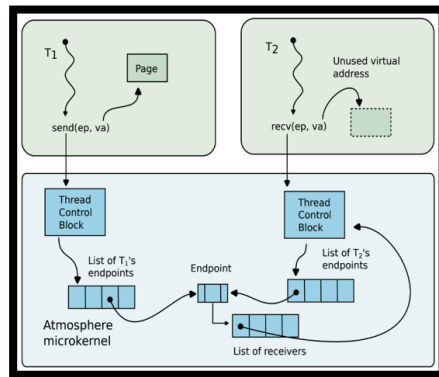
Concurrent
Memory Allocator



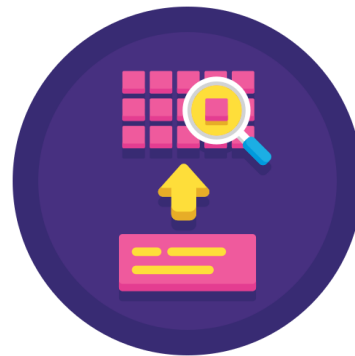
Persistent-
Memory Log



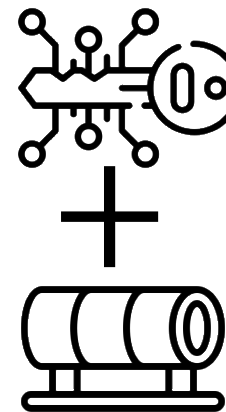
Microkernel



Serializer + Parser
Generator



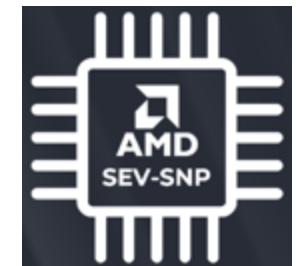
Crypto
Protocols



Kubernetes
Controller



Security Module for
Confidential VMs

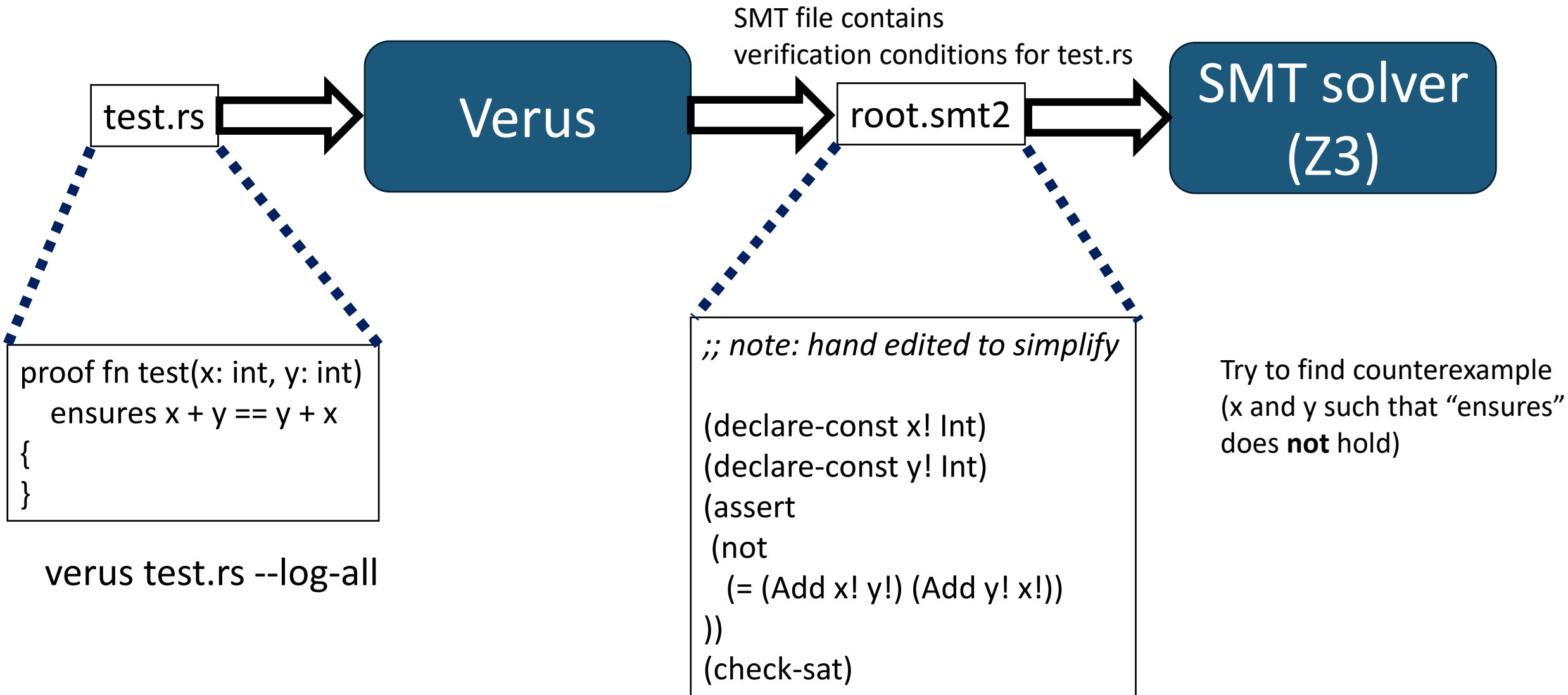


Outline of lectures

- Verification condition (VC) generation
- Using SMT solvers
- Higher-order features, vstd library
- Generics and traits (briefly)
- Ownership, &, &mut
- Verifying unsafe Rust code
- Concurrency

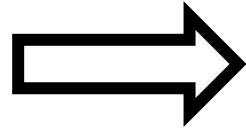
Verification condition (VC) generation

Verification condition generation



Verification condition generation

test.rs



root.air



root.smt2

```
proof fn test1(i: int) -> (r: int)
  ensures r == i + 15
{
  let mut x = i + 3;
  x = x + 5;
  x = x + 7;
  x
}
```

```
;; note: hand edited to simplify
(check-valid
 (declare-const r! Int)
 (declare-const i! Int)
 (declare-var x@ Int)

 (block
  (assume (= x@ (Add i! 3)))
  (assign x@ (Add x@ 5))
  (assign x@ (Add x@ 7))
  (assume (= r! x@))
  (assert (= r! (Add i! 15)))
 )
 )
```

```
;; note: hand edited to simplify
(declare-const r! Int)
(declare-const i! Int)
(declare-const x@0 Int)
(declare-const x@1 Int)
(declare-const x@2 Int)
(assert
 (not
  (=> (= x@0 (Add i! 3))
    (=> (= x@1 (Add x@0 5))
      (=> (= x@2 (Add x@1 7))
        (=> (= r! x@2)
          (= r! (Add i! 15))
        )
      )
    )
  )
)
)
)
)
)
(check-sat)
```

Weakest preconditions

From Speaking Logic lecture:

The proof of a valid triple $\{P\}S\{Q\}$ can be decomposed into

1. The valid triple $\{wlp(S)(Q)\}S\{Q\}$, and
2. The valid assertion $P \implies wlp(S)(Q)$

```
wlp({
  assume x = i + 3;
  assign x := x + 5;
  assign x := x + 7;
  assume r = x;
  assert r = i + 15
},
true)
```

```
wlp((assert E), Q) = E && Q
wlp((assume E), Q) = E ==> Q
wlp((assign x := E), Q) = subst x := E in Q
wlp((S1; S2), Q) = wlp(S1, wlp(S2, Q))
```

```
x = i + 3 ==>
r = (x + 5) + 7 ==>
r = i + 15 && true
```

```
wlp(assume x = i + 3,
wlp(assign x := x + 5,
wlp(assign x := x + 7,
wlp(assume r = x,
wlp(assert r = i + 15,
true))))))
```

```
wlp(assume x = i + 3,
wlp(assign x := x + 5,
wlp(assign x := x + 7,
wlp(assume r = x,
r = i + 15 && true
))))
```

```
wlp(assume x = i + 3,
wlp(assign x := x + 5,
wlp(assign x := x + 7,
r = x ==>
r = i + 15 && true
)))
```

```
wlp(assume x = i + 3,
wlp(assign x := x + 5,
r = x + 7 ==>
r = i + 15 && true
))
```

```
wlp(assume x = i + 3,
r = (x + 5) + 7 ==>
r = i + 15 && true
)
```

Weakest preconditions via SSA (static single assignment)

```
wlp({
  assume  $x_0 = i + 3$ ;
  assign  $x_1 := x_0 + 5$ ;
  assign  $x_2 := x_1 + 7$ ;
  assume  $r = x_2$ ;
  assert  $r = i + 15$ 
},
true)
```

$wlp((\text{assert } E), Q) = E \ \&\& \ Q$
 $wlp((\text{assume } E), Q) = E \implies Q$
 $wlp((\text{assign } x_n := E), Q) = x_n = E \implies Q$
 $wlp((S1; S2), Q) = wlp(S1, wlp(S2, Q))$

```
 $x_0 = i + 3 \implies$ 
 $x_1 = x_0 + 5 \implies$ 
 $x_2 = x_1 + 7 \implies$ 
 $r = x_2 \implies$ 
 $r = i + 15 \ \&\& \ \text{true}$ 
```

```
wlp(assume  $x_0 = i + 3$ ,
wlp(assign  $x_1 := x_0 + 5$ ,
wlp(assign  $x_2 := x_1 + 7$ ,
wlp(assume  $r = x_2$ ,
wlp(assert  $r = i + 15$ ,
true))))))
```

```
wlp(assume  $x_0 = i + 3$ ,
wlp(assign  $x_1 := x_0 + 5$ ,
wlp(assign  $x_2 := x_1 + 7$ ,
wlp(assume  $r = x_2$ ,
 $r = i + 15 \ \&\& \ \text{true}$ 
))))
```

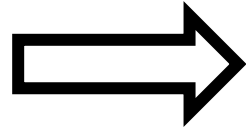
```
wlp(assume  $x_0 = i + 3$ ,
wlp(assign  $x_1 := x_0 + 5$ ,
wlp(assign  $x_2 := x_1 + 7$ ,
 $r = x_2 \implies$ 
 $r = i + 15 \ \&\& \ \text{true}$ 
)))
```

```
wlp(assume  $x_0 = i + 3$ ,
wlp(assign  $x_1 := x_0 + 5$ ,
 $x_2 = x_1 + 7 \implies$ 
 $r = x_2 \implies$ 
 $r = i + 15 \ \&\& \ \text{true}$ 
))
```

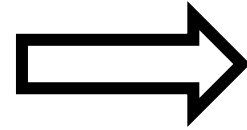
```
wlp(assume  $x_0 = i + 3$ ,
 $x_1 = x_0 + 5 \implies$ 
 $x_2 = x_1 + 7 \implies$ 
 $r = x_2 \implies$ 
 $r = i + 15 \ \&\& \ \text{true}$ 
)
```

Verification condition generation

test.rs



root.air



root.smt2

```
proof fn test1(i: int) -> (r: int)
  ensures r == i + 15
{
  let mut x = i + 3;
  x = x + 5;
  x = x + 7;
  x
}
```

```
;; note: hand edited to simplify
(check-valid
  (declare-const r! Int)
  (declare-const i! Int)
  (declare-var x@ Int)

  (block
    (assume (= x@ (Add i! 3)))
    (assign x@ (Add x@ 5))
    (assign x@ (Add x@ 7))
    (assume (= r! x@))
    (assert (= r! (Add i! 15)))
  )
)
```

```
;; note: hand edited to simplify
(declare-const r! Int)
(declare-const i! Int)
(declare-const x@0 Int)
(declare-const x@1 Int)
(declare-const x@2 Int)
(assert
  (not
    (=> (= x@0 (Add i! 3))
      (=> (= x@1 (Add x@0 5))
        (=> (= x@2 (Add x@1 7))
          (=> (= r! x@2)
            (= r! (Add i! 15))
          )
        )
      )
    )
  )
)
(check-sat)
```

```
x0 = i + 3 ==>
x1 = x0 + 5 ==>
x2 = x1 + 7 ==>
r = x2 ==>
r = i + 15 && true
```

Using SMT solvers

Decidable and undecidable theories

DECIDABLE THEORIES

boolean expressions

$e ::= \text{true} \mid \text{false} \mid !e \mid e \ \&\& \ e \mid e \implies e \mid \dots$

linear integer arithmetic

$e ::= \dots \mid -2 \mid -1 \mid 0 \mid 1 \mid 2 \mid e + e \mid e - e \mid e \leq e \mid \dots$

bit vector arithmetic

$e ::= \dots \mid e \ \& \ e \mid e \ll e \mid \dots$

uninterpreted functions

$e ::= \dots \mid f(e, \dots, e)$

datatypes

$e ::= \dots \mid S(e, \dots, e) \mid S.f \mid \text{match } e \{ \dots \}$

UNDECIDABLE EXTENSIONS TO ABOVE THEORIES

nonlinear integer arithmetic

$e ::= \dots \mid e * e \mid e / e \mid e \% e$

quantifiers

$e ::= \dots \mid (\text{forall } x: t \mid e) \mid (\text{exists } x: t \mid e)$

How can we use the decidable theories?

What techniques can help with the undecidable components?

SMT components used by Verus

Core Components (always enabled)

SAT solver

true | false | !e
e && e | ...

Linear int arith

-2 | -1 | 0 | 1 | 2
| e + e | e - e
| e <= e | ...

Uninterp funcs

spec fn f(x,...,x);
f(e,...,e)

Datatypes

struct S { ... }
enum E { ... }
S(e,...,e) | e.f | ...

Quantifiers

forall | x: t | e
exists | x: t | e

Additional Components (via “assert by”)

Bit vector arith

e & e | e << e |
...

Nonlin int arith

e * e
e / e | e % e

decidable, but can be slow

undecidable,
often unpredictable

- undecidable, but really important
- programmable by triggers

Congruence for uninterpreted functions

Uninterp funcs

```
spec fn f(x,...,x);  
f(e,...,e)
```

Congruence principle:
equal inputs imply equal output

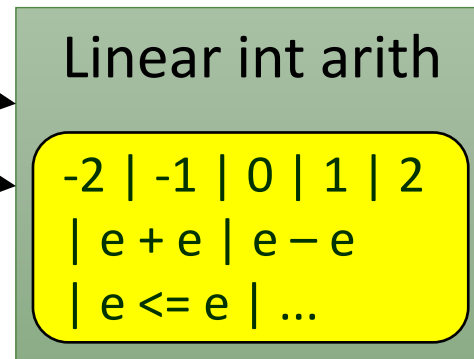
$$\left. \begin{array}{l} e1 == e1' \\ e2 == e2' \\ \dots \\ en == en' \end{array} \right\} \implies f(e1, \dots, en) == f(e1', \dots, en')$$

Example: uninterp funcs + arithmetic

```
struct MySeq { ... }
```

```
spec fn empty() -> MySeq;  
spec fn len(s: MySeq) -> nat;  
spec fn push(s: MySeq, value: int) -> MySeq;
```

```
proof fn test(x: int, y: int) {  
  let s = empty();  
  assert(push(s, x + y)  
    == push(s, y + x));  
}
```



Uninterp funcs

spec fn f(x,...,x);
f(e,...,e)

$x + y == y + x$

$\text{push}(s, x + y) == \text{push}(s, y + x)$

Interpreted functions (nonrecursive)

Core Components (always enabled)

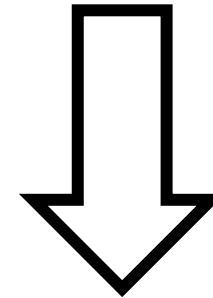
Uninterp funcs

```
spec fn f(x,...,x);  
f(e,...,e)
```

Quantifiers

```
forall |x: t| e  
exists |x: t| e
```

```
spec fn f(x: int, y: int) -> int {  
  x + 2 * y  
}
```



```
spec fn f(x: int, y: int) -> int;
```

axiom:

```
forall |x: int, y: int|  
  #[trigger] f(x, y) == x + 2 * y
```

Interpreted functions (recursive)

Core Components (always enabled)

Uninterp funcs

```
spec fn f(x,...,x);  
f(e,...,e)
```

Quantifiers

```
forall |x: t| e  
exists |x: t| e
```

```
spec fn f(x: int) -> bool  
  decreases x  
{  
  x <= 0 || !f(x - 1)  
}
```

Naive axiom generates endless instantiations:

```
f(x)  
== x <= 0 || !f(x - 1)  
== x <= 0 || !(x - 1 <= 0 || !f(x - 1 - 1))  
== x <= 0 || !(x - 1 <= 0 || !(x - 1 - 1 <= 0 || !f(...)))  
...
```

Verus axiom uses “fuel” to limit unrolling depth

```
proof fn test() {  
  reveal_with_fuel(f, 4);  
  assert(!f(3));  
}
```

Assert-by for bit vector, nonlinear arith

Core Components (always enabled)

SAT solver

true | false | !e
e && e | ...

Linear int arith

-2 | -1 | 0 | 1 | 2
| e + e | e - e
| e <= e | ...

Uninterp funcs

spec fn f(x,...,x);
f(e,...,e)

Datatypes

struct S { ... }
enum E { ... }
S(e,...,e) | e.f | ...

Quantifiers

forall | x: t | e
exists | x: t | e

Additional Components (via “assert by”)

Bit vector arith

e & e | e << e |
...

Nonlin int arith

e * e
e / e | e % e

```
proof fn test_by(x: u32)
  requires x <= 200,
{
  assert(x >> 8 == 0) by(bit_vector)
  requires(x <= 255);
}
```

Assert-by for bit vector, nonlinear arith

Core Components (always enabled)

SAT solver

true | false | !e
e && e | ...

Linear int arith

-2 | -1 | 0 | 1 | 2
| e + e | e - e
| e <= e | ...

Uninterp funcs

spec fn f(x,...,x);
f(e,...,e)

Datatypes

struct S { ... }
enum E { ... }
S(e,...,e) | e.f | ...

Quantifiers

forall | x: t | e
exists | x: t | e

Additional Components (via “assert by”)

Bit vector arith

e & e | e << e |
...

Nonlin int arith

e * e
e / e | e % e

```
proof fn test_by(x: u32, y: u32)
  requires x > 1 && y > 0
{
  assert(x * y > y) by(nonlinear_arith)
  requires(x > 1 && y > 0);
}
```

Quantifiers

```
spec fn f(i: int) -> int;  
spec fn g(i: int) -> int;
```

```
// hard case: using forall  
proof fn test()  
  requires  
    forall |x: int, y: int| f(x) > g(y)  
  ensures  
    ...  
{ ... }
```

Quantifiers

```
forall |x: t| e  
exists |x: t| e
```

```
// easy case: proving forall  
proof fn test()  
  requires  
    ...  
  ensures  
    forall |x: int, y: int| f(x) > g(y)  
{ ... }
```

```
proof fn test(x: int, y: int)  
  requires  
    ...  
  ensures  
    f(x) > g(y)  
{ ... }
```

Quantifiers

```
spec fn f(i: int) -> int;  
spec fn g(i: int) -> int;
```

```
// hard case: using forall  
proof fn test()  
  requires  
    forall |x: int, y: int| f(x) > g(y)  
  ensures  
    f(g(7)) > g(3)  
{  
}
```

Quantifiers

forall |x: t| e
exists |x: t| e

Quantifiers

Quantifiers

forall $|x: t| e$
exists $|x: t| e$

$(\text{forall } |x: \text{int}, y: \text{int}| f(x) > g(y))$
 $\implies f(g(7)) > g(3)$

Infinitely many possible instantiations of x, y

...

$f(2) > g(7)$

$f(4) > g(6)$

$f(4) > g(7)$

$f(5) > g(10)$

$f(5) > g(-5)$

...

Quantifiers

Quantifiers

forall |x: t| e
exists |x: t| e

pattern ("trigger") to match

(forall |x: int, y: int| #[trigger f(x), g(y)] f(x) > g(y))
==> f(g(7)) > g(3)

Quantifiers

Quantifiers

forall $|x: t| e$
exists $|x: t| e$

(alternate trigger notation for trigger $f(x), g(y)$)

(forall $|x: \text{int}, y: \text{int}| \text{ \#}[trigger] f(x) > \text{ \#}[trigger] g(y)$)
 $\implies f(g(7)) > g(3)$

Quantifiers

Quantifiers

forall $|x: t| e$
exists $|x: t| e$

pattern ("trigger") to match

(forall $|x: \text{int}, y: \text{int}|$ #![trigger f(x), g(y)] $f(x) > g(y)$)
 $\implies f(g(7)) > g(3)$

matches
 $g(y)$ with
 $y = 7$
and
 $f(x)$ with
 $x = g(7)$

matches
 $g(y)$ with
 $y = 3$

Particular instantiations of x, y
based on trigger match

$f(g(7)) > g(3)$
 $f(g(7)) > g(7)$

Quantifiers

Quantifiers

forall |x: t| e
exists |x: t| e

pattern (“trigger”) to match

(forall |x: int, y: int| #[trigger f(x), g(y)] f(x) > g(y))
==> f(g(7)) > g(3)

Triggers may be written by user,
or chosen automatically by Verus.

Quantifiers

Quantifiers

forall | x: t | e
exists | x: t | e

pattern ("trigger") to match

(forall | x: int, y: int | #[trigger f(x), g(y)] f(x) > g(y))
==> f(g(7)) > g(3)

- The trigger must mention all the quantified variables
 - both x and y
- The trigger matches only if **all** of its patterns match
 - both f(x) and g(y) must match some expressions

Quantifiers

Quantifiers

forall $|x: t| e$
exists $|x: t| e$

badly behaved trigger

(forall $|x: \text{int}|$ *#[trigger f(x)]* $f(f(x)) > f(x)$)
 $\implies f(3) > f(f(2))$

Particular instantiations of x, y
based on trigger match

Beware of infinite matching loops

Quantifiers

Quantifiers

forall |x: t| e
exists |x: t| e

badly behaved trigger

(forall |x: int| *#[trigger f(x)]* f(f(x)) > f(x))
==> f(3) > f(f(2))

Particular instantiations of x, y
based on trigger match

f(f(3)) > f(3)

f(f(f(3))) > f(f(3))

f(f(f(f(3)))) > f(f(f(3)))

...

Beware of infinite matching loops

Quantifiers

Quantifiers

forall $|x: t| e$
exists $|x: t| e$

well behaved trigger

(forall $|x: \text{int}|$ *#[trigger f(f(x))]* $f(f(x)) > f(x)$)
 $\implies f(3) > f(f(2))$

Particular instantiations of x, y
based on trigger match

$f(f(2)) > f(2)$

Triggers are the programming language for quantifiers.
Choose triggers carefully!

Higher-order features,
vstd library

Higher-order features: spec_fn

```
proof fn test() {  
  let f: spec_fn(int, int) -> int = |x: int, y: int| x + x - y;  
  let g: spec_fn(int, int) -> int = |x: int, y: int| 2 * x - y;  
  assert(f(30, 20) == 40);  
  assert(forall |x: int, y: int| f(x, y) == g(x, y));  
}
```

SMT encoding (simplified):

```
(declare-sort Fun)  
(declare-const f Fun)  
(declare-const g Fun)  
(declare-fun apply (Fun Int Int) Int)  
(axiom (forall ((x Int) (y Int)) (= (apply f x y) (Sub (Add x x) y))))  
(axiom (forall ((x Int) (y Int)) (= (apply g x y) (Sub (Mul 2 x) y))))
```

Note: real SMT encoding includes various types/arities, closure conversion, and hashing of similar closures

Verus's vstd library: Seq, Set, Map, etc.

// finite sequence

```
struct Seq<A> { ... }
```

```
impl<A> Seq<A> {
```

```
  spec fn empty() -> Seq<A>;
```

```
  spec fn new(len: nat, f: spec_fn(int) -> A) -> Seq<A>;
```

```
  spec fn len(self) -> nat;
```

```
  spec fn index(self, i: int) -> A;
```

```
  spec fn push(self, a: A) -> Seq<A>;
```

```
  ...
```

```
}
```

```
proof fn test() {
```

```
  let s: Seq<int> = Seq::new(5, |i: int| 10 * i);
```

```
  assert(s.len() == 5);
```

```
  assert(s.index(3) == 30); // syntax sugar: s[3] == 30
```

```
  let s2 = s.push(50);
```

```
  assert(s2.len() == 6);
```

```
}
```

*Rust feature note:
"impl" and "self" are for
namespacing and "." syntax*

// finite or infinite set

```
struct Set<A> { set: spec_fn(A) -> bool }
```

```
impl<A> Set<A> {
```

```
  spec fn empty() -> Set<A> { Set { set: |a: A| false } }
```

```
  spec fn new(f: spec_fn(A) -> bool) -> Set<A> { Set { set: f } }
```

```
  spec fn contains(self, a: A) -> bool { (self.set)(a) }
```

```
  spec fn insert(self, a: A) -> Set<A> { ... }
```

```
  ...
```

```
}
```

// finite or infinite map

```
struct Map<K, V> { mapping: spec_fn(K) -> Option<V> }
```

```
impl<K, V> Map<K, V> {
```

```
  spec fn empty() -> Map<K, V> { Map { mapping: |k| None } }
```

```
  spec fn insert(self, key: K, value: V) -> Map<K, V> { ... }
```

```
  ...
```

```
}
```

Axioms and lemmas via function calls

```
proof fn axiom_seq_push_len<A>(s: Seq<A>, a: A)
  ensures
    s.push(a).len() == s.len() + 1
{ ... }
```

```
proof fn test() {
  let s: Seq<int> = Seq::new(5, |i: int| 10 * i);
  let s2 = s.push(50);
  axiom_seq_push_len(s, 50);
  assert(s2.len() == 6);
}
```

Axioms and lemmas via broadcast

```
broadcast proof fn axiom_seq_push_len<A>(s: Seq<A>, a: A)
  ensures
    #[trigger] s.push(a).len() == s.len() + 1
{ ... }
```

```
proof fn test() {
  broadcast use axiom_seq_push_len;
```

```
  let s: Seq<int> = Seq::new(5, |i: int| 10 * i);
  let s2 = s.push(50);
  assert(s2.len() == 6);
}
```

forall |s: Seq<A>, a: A | **#![trigger s.push(a).len()]**
s.push(a).len() == s.len() + 1

Broadcast groups and defaults

```
struct Seq<A> { ... }  
impl<A> Seq<A> { ... }  
broadcast proof fn axiom_seq_push_len<A>(...) ...  
...
```

```
broadcast group group_seq_axioms {  
  axiom_seq_push_len,  
  axiom_seq_push_index_same,  
  axiom_seq_push_index_different,  
  ...  
}
```

```
#[verifier::broadcast_use_by_default_when_this_crate_is_imported]  
broadcast group group_vstd_default {  
  group_seq_axioms,  
  group_set_axioms,  
  group_map_axioms,  
  ...  
}
```

Verus vstd crate

import vstd crate

user crate

```
use vstd::prelude::*;  
  
proof fn test() {  
  let s: Seq<int> = Seq::new(5, |i: int| 10 * i);  
  let s2 = s.push(50);  
  assert(s2.len() == 6);  
}
```

Equality via extensionality

```
proof fn test() {  
  let s1 = seq![10] + seq![20, 30];  
  let s2 = seq![10, 20] + seq![30];  
  // Does SMT solver know s1 == s2?  
  
}
```

Seq::add([10], [20, 30]) ==
Seq::add([10, 20], [30])

Uninterp funcs

spec fn f(x,...,x);
f(e,...,e)

congruence
doesn't
help here!

[10] == [10, 20]
[20, 30] == [30]

Equality via extensionality

```
broadcast proof fn axiom_seq_ext_equal<A>(s1: Seq<A>, s2: Seq<A>)
```

```
  ensures
```

```
    #[trigger] (s1 ==~ s2) <==>
```

```
      s1.len() == s2.len() && (forall |i: int| 0 <= i < s1.len() ==> s1[i] == s2[i])
```

```
{ ... }
```

```
proof fn test() {
```

```
  let s1 = seq![10] + seq![20, 30];
```

```
  let s2 = seq![10, 20] + seq![30];
```

```
  // Does SMT solver know s1 == s2?
```

```
  // ==~ is semantically equivalent to ==, but will trigger additional lemmas
```

```
  assert(s1 ==~ s2); // triggers axiom_seq_ext_equal
```

```
  // Now SMT solver knows s1 == s2
```

```
}
```

```
// Note: Verus automatically replaces == with ==~ in assert, ensures, invariant
```

Decidable and undecidable theories

DECIDABLE THEORIES

boolean expressions

$e ::= \text{true} \mid \text{false} \mid !e \mid e \ \&\& \ e \mid e \implies e \mid \dots$

linear integer arithmetic

$e ::= \dots \mid -2 \mid -1 \mid 0 \mid 1 \mid 2 \mid e + e \mid e - e \mid e \leq e \mid \dots$

bit vector arithmetic

$e ::= \dots \mid e \ \& \ e \mid e \ll e \mid \dots$

uninterpreted functions

$e ::= \dots \mid f(e, \dots, e)$

datatypes

$e ::= \dots \mid S(e, \dots, e) \mid S.f \mid \text{match } e \{ \dots \}$

UNDECIDABLE EXTENSIONS TO ABOVE THEORIES

nonlinear integer arithmetic

$e ::= \dots \mid e * e \mid e / e \mid e \% e$

quantifiers

$e ::= \dots \mid (\text{forall } x: t \mid e) \mid (\text{exists } x: t \mid e)$

Verus uses quantifiers for many broadcast lemmas, including $\approx$,
so we can't expect completely automated proofs

Decidability, undecidability, automation

- For decidable theories, we can hope for complete SMT automation
 - ... except when the SMT decision procedures are too slow
 - `assert(...)` by `bit_vector`
- But most Verus code relies on features beyond the decidable fragment
 - Nonlinear arithmetic: `assert(...)` by `nonlinear_arith`
 - Verus's lemmas for Seq, Set, Map, etc.
 - Equality via extensionality (`=~=`)
 - User-supplied quantifiers
 - `forall |i: int, j: int| 0 <= i <= j < v.len() ==> v[i] <= v[j]`
 - Recursive definitions and proofs by induction

So we get some mixture of automation and explicit proofs

Outline of lectures

- Verification condition (VC) generation
- Using SMT solvers
- Higher-order features, vstd library
- Generics and traits (briefly)
- Ownership, &, &mut
- Verifying unsafe Rust code
- Concurrency

Generics and traits (briefly)

Rust feature: traits (aka Haskell type classes)

```
trait Clone {  
    fn clone(&self) -> Self;  
}  
  
trait Copy {}  
  
trait Default {  
    fn default() -> Self;  
}  
  
trait Add<Rhs> {  
    type Output;  
    fn add(self, rhs: Rhs) -> Self::Output;  
}
```

```
struct Complex { real: f64, imag: f64 }  
impl Clone for Complex {  
    fn clone(&self) -> Complex {  
        Complex { real: self.real, imag: self.imag }  
    }  
}  
impl Default for Complex {  
    fn default() -> Complex {  
        Complex { real: 0.0, imag: 0.0 }  
    }  
}  
impl Add<Complex> for Complex {  
    type Output = Complex;  
    fn add(self, rhs: Complex) -> Complex {  
        Complex {  
            real: self.real + rhs.real,  
            imag: self.imag + rhs.imag,  
        }  
    }  
}
```

```
fn f<T>(x: T) -> T::Output  
where  
    T: Default,  
    T: Add<T>,  
{  
    x + T::default()  
}  
fn test() {  
    let c1 = Complex {  
        real: 3.14,  
        imag: 2.72,  
    };  
    let c2 = Complex::default();  
    let c3 = c1.clone();  
    let c4 = c1 + c2 + c3;  
    let c5 = f(c4);  
}
```

Notes:

- the "self" parameter uses "." syntax
- "impl S for Trait { ... }" is a trait impl
- "impl S { ... }" is for namespacing and "." syntax, unrelated to traits

Verus View trait: exec value -> spec value

```
trait View {  
  type V;  
  spec fn view(&self) -> Self::V;  
}  
  
impl View for String {  
  type V = Seq<char>;  
  spec fn view(&self) -> Seq<char> { ... }  
}  
  
impl<T> View for Vec<T> {  
  type V = Seq<T>;  
  spec fn view(&self) -> Seq<T> { ... }  
}  
  
impl<T> View for HashSet<T> {  
  type V = Set<T>;  
  spec fn view(&self) -> Set<T> { ... }  
}  
  
impl<Key, Value> View for HashMap<Key, Value> {  
  type V = Map<Key, Value>;  
  spec fn view(&self) -> Map<Key, Value> { ... }  
}
```

```
fn test() {  
  let mut v: Vec<u8> = Vec::new();  
  v.push(10);  
  v.push(20);  
  v.push(30);  
  proof {  
    let s: Seq<u8> = v@;  
    assert(s == seq![10, 20, 30]);  
  }  
}
```

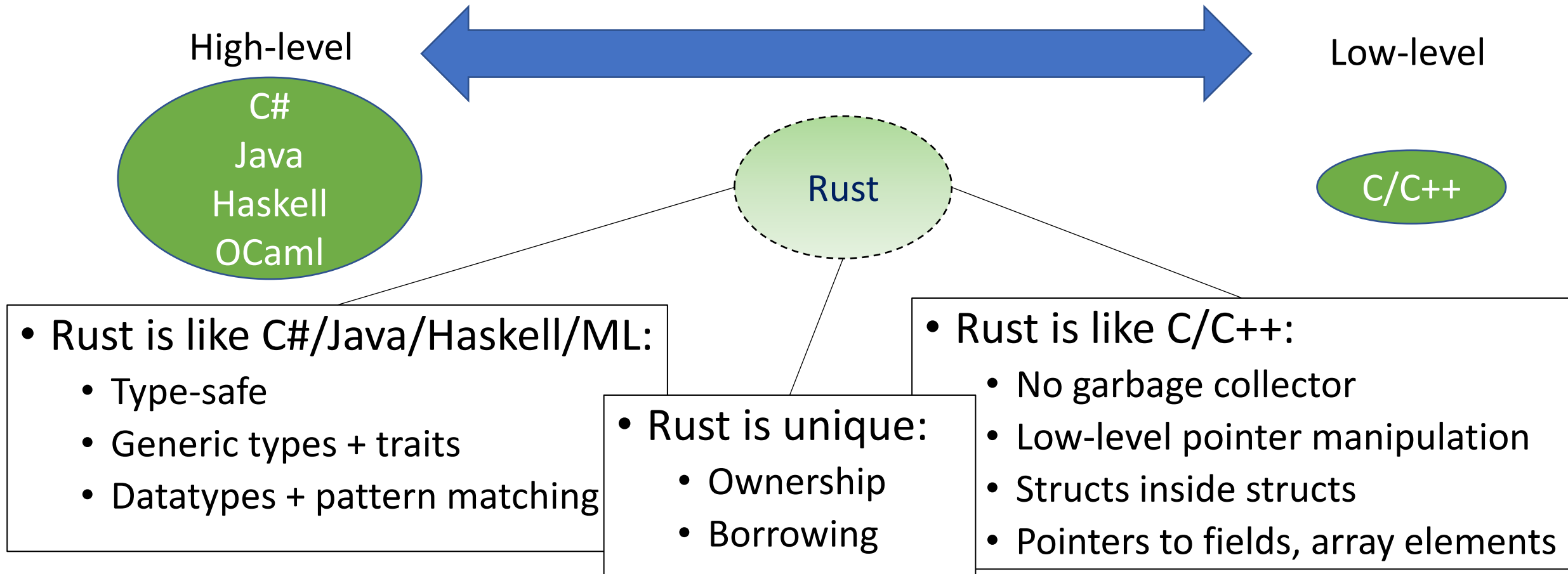
“v@” is syntax sugar for “v.view()”

```
fn test() {  
  let mut v: Vec<u8> = Vec::new();  
  v.push(10);  
  v.push(20);  
  v.push(30);  
  let ghost s: Seq<u8> = v@;  
  assert(s == seq![10, 20, 30]);  
}
```

“ghost”: proof-only variable

Ownership, &, &mut

Why write systems code in Rust?



Ownership in Rust

```
fn consume(v: Vec<u8>) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    v.push(10);  
    consume(v);  
    v.push(20); // ERROR: v no longer available  
}
```

Note: types implementing Copy (bool, u8, ...) are copied, not moved
Vec does *not* implement Copy, so it is moved, not copied

Borrowing in Rust

```
fn borrow(v: &Vec<u8>) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    v.push(10);  
    borrow(&v);  
    v.push(20); // ok  
}
```

```
fn borrow(v: &mut Vec<u8>) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    v.push(10);  
    borrow(&mut v);  
    v.push(20); // ok  
}
```

Borrowing in Rust

```
fn borrow(  
    v1: &Vec<u8>,  
    v2: &Vec<u8>,  
) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    borrow(&v, &v); // ok  
}
```

```
fn borrow(  
    v1: &Vec<u8>,  
    v2: &mut Vec<u8>,  
) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    borrow(&v, &mut v); // error  
}
```

```
fn borrow(  
    v1: &mut Vec<u8>,  
    v2: &mut Vec<u8>,  
) {  
}  
  
fn test() {  
    let mut v = Vec::new();  
    borrow(&mut v, &mut v); // error  
}
```

multiple shared (immutable) references to v simultaneously (aliasing ok),
or
one mutable reference allowed to v (no aliasing)

Note: &t implements Copy, while &mut t does *not* implement Copy

Mutation in Rust

```
fn f(v: Vec<u8>) {  
    v.push(10); // ok  
}
```

```
fn f(v: &Vec<u8>) {  
    v.push(10); // error  
}
```

```
fn f(v: &mut Vec<u8>) {  
    v.push(10); // ok  
}
```

- mutation allowed on owned value and mutable reference
- shared references does not allow mutation

So Rust allows ***either mutation or aliasing*** of a value,
but not both mutation and aliasing together.

This is an unusual discipline.

Most languages (C, C++, Java, C#, ML, JavaScript, Python, Dafny, ...) allow mutation and aliasing together.

Mutation + aliasing is annoying for verification

// Dafny code

method M(x: array<int>, y: array<int>)

requires x.Length > 0

requires y.Length > 0

requires y[0] == 3

modifies x

{

assert y[0] == 3;

x[0] := 5;

assert y[0] == 3; // error

}

// Dafny code

method M(x: array<int>, y: array<int>)

requires x != y

requires x.Length > 0

requires y.Length > 0

requires y[0] == 3

modifies x

{

assert y[0] == 3;

x[0] := 5;

assert y[0] == 3; // ok

}

// Rust/Verus

fn f(x: &mut Vec<u8>, y: &Vec<u8>)

requires

x.len() > 0,

y.len() > 0,

y[0] == 3,

{

assert(y[0] == 3);

x[0] = 5;

assert(y[0] == 3); // ok

}

Classical verification (e.g. Dafny): one big heap

```
// Dafny code
```

```
method M(x: array<int>, y: array<int>)  
  requires x != y
```

Classical approach:

- x and y are addresses
- objects (e.g. arrays) live in heap
- need to worry about aliasing

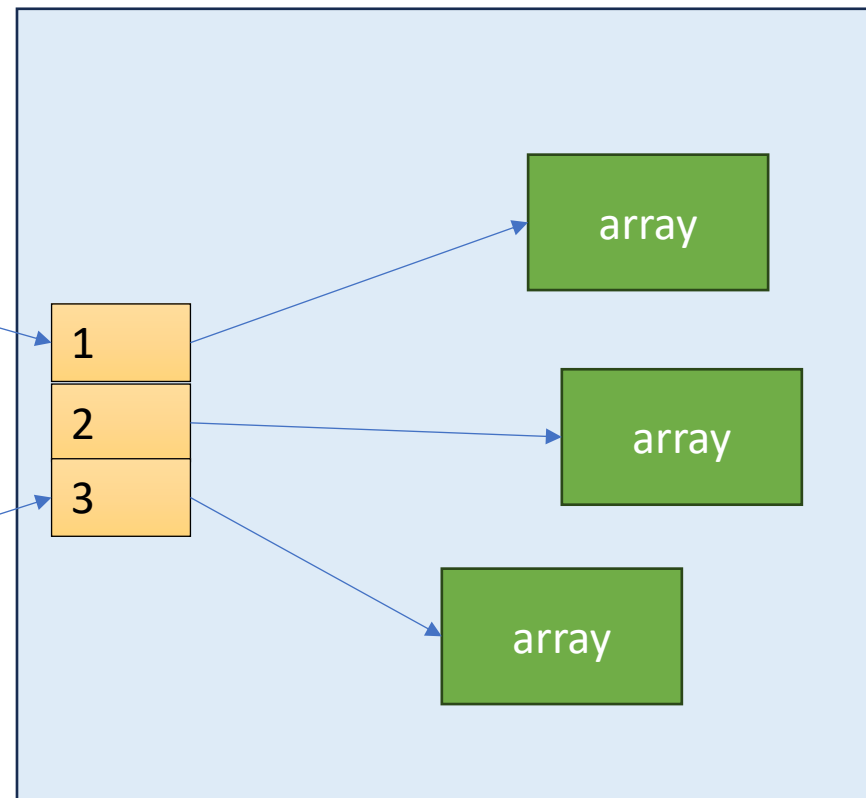
(declare-const y! Int)

y = 1

(declare-const x! Int)

x = 3

(declare-var heap@ MapIntToObject)



Rust verification: no heap needed

In AIR/SMT encoding, values are stored in the references, not in a separate heap:

- SMT representation of `&t` is the same as SMT representation of `t`
- SMT representation of `&mut t` is a pair of `t` and extra information

Let $[[t]]$ be the SMT representation of Rust type `t`:

$[[\text{bool}]] = \text{Bool}$

$[[\text{int}]] = \text{Int}$

$[[\&t]] = [[t]]$

$[[\&\text{mut } t]] = ([[t]], \dots \text{extra-stuff} \dots)$

$[[\text{Complex}]] = \text{Complex}$

...

```
struct Complex { real: f64, imag: f64 }
```

```
fn f(x: Complex, y: &Complex, z: &mut Complex) { ... }
```

(declare-const x! Complex)

(declare-const y! Complex)

(declare-const z! ...)

Rust &mut is powerful and complex

```
struct S { a: u32, b: u32, c: u32 }
```

```
fn f(uref: &mut u32) {  
    *uref += 1;  
}  
  
fn g() {  
    let mut s = S { a: 10, b: 20, c: 30 };  
    f(&mut s.c);  
    assert!(s == S { a: 10, b: 20, c: 31 });  
}
```

```
fn f(sref: &mut S) -> &mut u32 {  
    &mut sref.c  
}  
  
fn g() {  
    let mut s = S { a: 10, b: 20, c: 30 };  
    let p = f(&mut s);  
    *p = *p + 1;  
    assert!(s == S { a: 10, b: 20, c: 31 });  
}
```

```
fn f(flag: bool, sref: &mut S) -> &mut u32 {  
    if flag { &mut sref.b } else { &mut sref.c }  
}  
  
fn g() {  
    let mut s = S { a: 10, b: 20, c: 30 };  
    let p = f(true, &mut s);  
    *p = *p + 1;  
    assert!(s == S { a: 10, b: 21, c: 30 });  
}
```

[[&mut t]] = ([[t]], ...extra-stuff...)

The “extra-stuff” says how to put the new [[t]] into the place the old [[t]] was borrowed from

SMT encoding of &mut

$[[\&\text{mut } t]] = ([[t]], \dots\text{extra-stuff}\dots)$

The “extra-stuff” says how to put the new $[[t]]$ into the place the old $[[t]]$ was borrowed from

```
fn f(uref: &mut u32) {  
    *uref += 1;  
}  
  
fn g() {  
    let mut s = S { a: 10, b: 20, c: 30 };  
    f(&mut s.c);  
    assert!(s == S { a: 10, b: 20, c: 31 });  
}
```

Lots of research in last 10 years, various approaches

- “extra-stuff” could be address (but then we’d need a heap, which we don’t want)
- “extra-stuff” could be a function generating new S from new $[[t]]$
- “extra-stuff” could be a function saying what’s true about new S , given new $[[t]]$

Verus (following RustHorn, Creusot) uses “prophecies”:

- “RustHorn: CHC-based Verification for Rust Programs”
(Matsushita, Tsukada, and Kobayashi, ESOP 2020)
“extra-stuff” is a “prophecy” of type $[[t]]$ naming the final value of $[[t]]$

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized final value)

SMT encoding of &mut

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized final value)

```
fn f(uref: &mut u32) {  
    *uref += 1;  
}  
  
fn g() {  
    let mut s = S { a: 10, b: 20, c: 30 };  
    f(&mut s.c);  
    assert!(s == S { a: 10, b: 20, c: 31 });  
}
```

$s = S \{ a: 10, b: 20, c: 30 \}$

(declare-const X Int)

**Let X be the name of the prophesized final value of &mut s.c
Immediately replace old value (30) with X
 $s = S \{ a: 10, b: 20, c: X \}$**

$uref = (30, X)$

(assign s (... (c X)...))

$uref = (31, X)$

**At the end of f, the &mut expires (it is owned by f, and &mut is not Copy)
When the &mut expires, we know its final value: $X = 31$
 $uref = (31, 31)$**

**Knowing $X = 31$ allows g to conclude:
 $s = S \{ a: 10, b: 20, c: 31 \}$**

(assume (= (...prophecy...) (...current...)))

SMT encoding of &mut

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized final value)

```
fn f(uref: &mut u32)
  requires
    *uref < u32::MAX,
  ensures
    *final(uref) == *old(uref) + 1,
  {
    *uref += 1;
  }

fn g() {
  let mut s = S { a: 10, b: 20, c: 30 };
  f(&mut s.c);
  assert!(s == S { a: 10, b: 20, c: 31 });
}
```

$s = S \{ a: 10, b: 20, c: 30 \}$

Let X be the name of the prophesized final value of $\&\text{mut } s.c$
Immediately replace old value (30) with X
 $s = S \{ a: 10, b: 20, c: X \}$

$\text{uref} = (\text{30}, X)$

$\text{uref} = (31, X)$

At the end of f , the $\&\text{mut}$ expires (it is owned by f , and $\&\text{mut}$ is not Copy)
When the $\&\text{mut}$ expires, we know its final value: $X = 31$
 $\text{uref} = (31, 31)$

Knowing $X = 31$ allows g to conclude:
 $s = S \{ a: 10, b: 20, c: 31 \}$

SMT encoding of &mut

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized)

```
fn f(sref: &mut S) -> (r: &mut u32) {  
  &mut sref.c  
}  
  
fn g() {  
  let mut s = S { a: 10, b: 20, c: 30 };  
  let p = f(&mut s);  
  *p = *p + 1;  
  assert!(s == S { a: 10, b: 20, c: 31 });  
}
```

$s = S \{ a: 10, b: 20, c: 30 \}$

Let X be the name of the prophesized final value of &mut s
Immediately replace old value (S { a: 10, b: 20, c: 30}) with X
 $s = X$

$sref = (S \{ a: 10, b: 20, c: 30 \}, X)$

Let Y be the name of the prophesized final value of &mut sref.c
 $sref = (S \{ a: 10, b: 20, c: Y \}, X)$
 $r = (30, Y)$
At the end of f, sref expires with $X = S \{ a: 10, b: 20, c: Y \}$

$p = (30, Y)$

$p = (31, Y)$

p expires with $Y = 31$

Knowing $X = S \{ a: 10, b: 20, c: Y \}$ and $Y = 31$
allows g to conclude: $s = S \{ a: 10, b: 20, c: 31 \}$

SMT encoding of &mut

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized final value)

```
fn f(sref: &mut S) -> (r: &mut u32)
  ensures
    *r == old(sref).c,
    final(sref).a == old(sref).a,
    final(sref).b == old(sref).b,
    final(sref).c == *final(r),
  {
    &mut sref.c
  }

fn g() {
  let mut s = S { a: 10, b: 20, c: 30 };
  let p = f(&mut s);
  *p = *p + 1;
  assert!(s == S { a: 10, b: 20, c: 31 });
}
```

$s = S \{ a: 10, b: 20, c: 30 \}$

Let X be the name of the prophesized final value of &mut s
Immediately replace old value ($S \{ a: 10, b: 20, c: 30 \}$) with X
 $s = X$

$sref = (S \{ a: 10, b: 20, c: 30 \}, X)$

Let Y be the name of the prophesized final value of &mut sref.c
 $sref = (S \{ a: 10, b: 20, c: Y \}, X)$
 $r = (30, Y)$

At the end of f, sref expires with $X = S \{ a: 10, b: 20, c: Y \}$

$p = (30, Y)$

$p = (31, Y)$

p expires with $Y = 31$

Knowing $X = S \{ a: 10, b: 20, c: Y \}$ and $Y = 31$
allows g to conclude: $s = S \{ a: 10, b: 20, c: 31 \}$

SMT encoding of &mut

$[[\&\text{mut } t]] = (\quad [[t]], \quad [[t]] \quad)$
(current value, prophesized final value)

```
fn f(flag: bool, sref: &mut S) -> (r: &mut u32)
  ensures
    *r == if flag { old(sref).b } else { old(sref).c },
    final(sref).a == old(sref).a,
    final(sref).b == if flag { *final(r) } else { old(sref).b },
    final(sref).c == if !flag { *final(r) } else { old(sref).c },
  {
    if flag { &mut sref.b } else { &mut sref.c }
  }

fn g() {
  let mut s = S { a: 10, b: 20, c: 30 };
  let p = f(true, &mut s);
  *p = *p + 1;
  assert(s == S { a: 10, b: 21, c: 30 });
}
```

Prophecy and soundness

Naive rules for prophecy can lead to unsoundness.

Example:

Let X be the name of a prophesized variable:

(declare-const X Int)

Resolve X to the expression (Add X 1)

(assume (= X (Add X 1)))

contradiction!

To prevent this, Verus limits use of prophetic variables to a few places like requires, ensures, assert, and specially marked prophetic spec fns

Verifying Unsafe Rust code

Separation logic / Rust ownership

Verus uses:

- Rust ownership to express ***separation*** (non-aliasing) of data
- SMT solvers (and higher-order extensions) to express ***logic***

Separation logic combines these and is the basis for much recent research in verification, particularly:

- unsafe code
- concurrent code (e.g. Iris)

However, SMT solvers don't directly support separation logic (they have **&&**, **| |**, **=>** built in, not *****).

As an alternative, we can import separation logic ideas into Verus using Rust ownership and SMT-based logic.

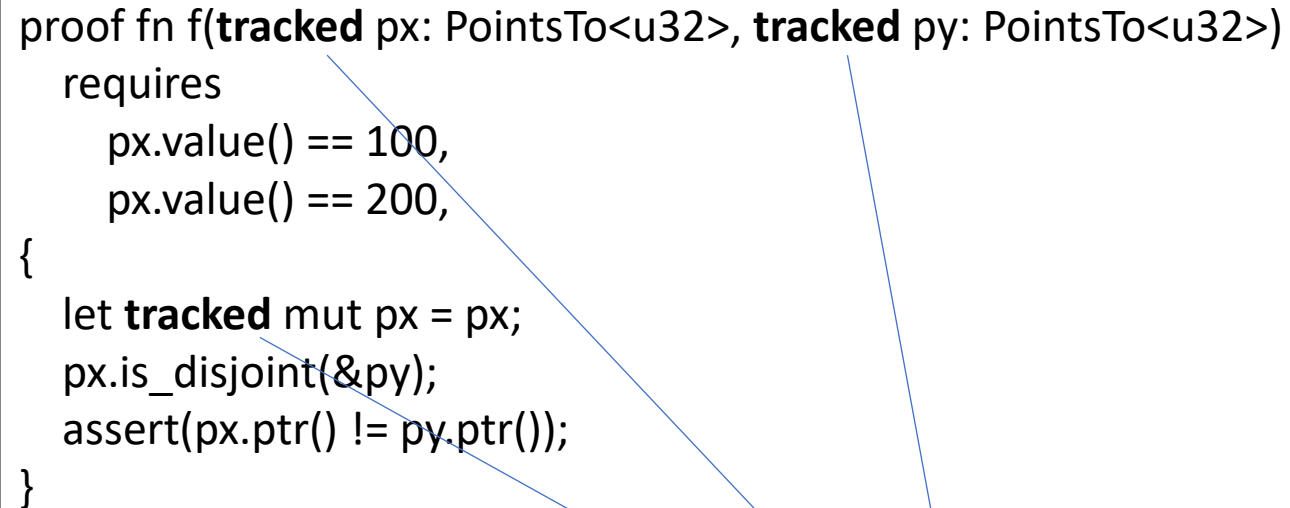
Separation Logic $\mapsto$ and $*$ is like “tracked PointsTo” and `,` in Verus

Separation logic pseudo-code

```
proof fn f(x: addr, y: addr)
  requires x  $\mapsto$  100 * y  $\mapsto$  200
{
  assert(x != y);
}
```

Actual Verus code for Rust unsafe raw pointers (using `vstd::raw_ptr::*`)

```
proof fn f(tracked px: PointsTo<u32>, tracked py: PointsTo<u32>)
  requires
    px.value() == 100,
    px.value() == 200,
{
  let tracked mut px = px;
  px.is_disjoint(&py);
  assert(px.ptr() != py.ptr());
}
```



tracked variables in proof code are:

- checked for ownership (and thus are “separated” like “`*`”)
- unforgeable (can’t just make up an arbitrary value of any type `t`)

Separation Logic $\mapsto$ and $*$ is like “tracked PointsTo” and , in Verus

Separation logic pseudo-code

```
fn f(x: addr, y: addr)
  requires x  $\mapsto$  100 * y  $\mapsto$  200
{
  let n = *x;
  *x = n + 1;
  assert(x  $\mapsto$  101);
}
```

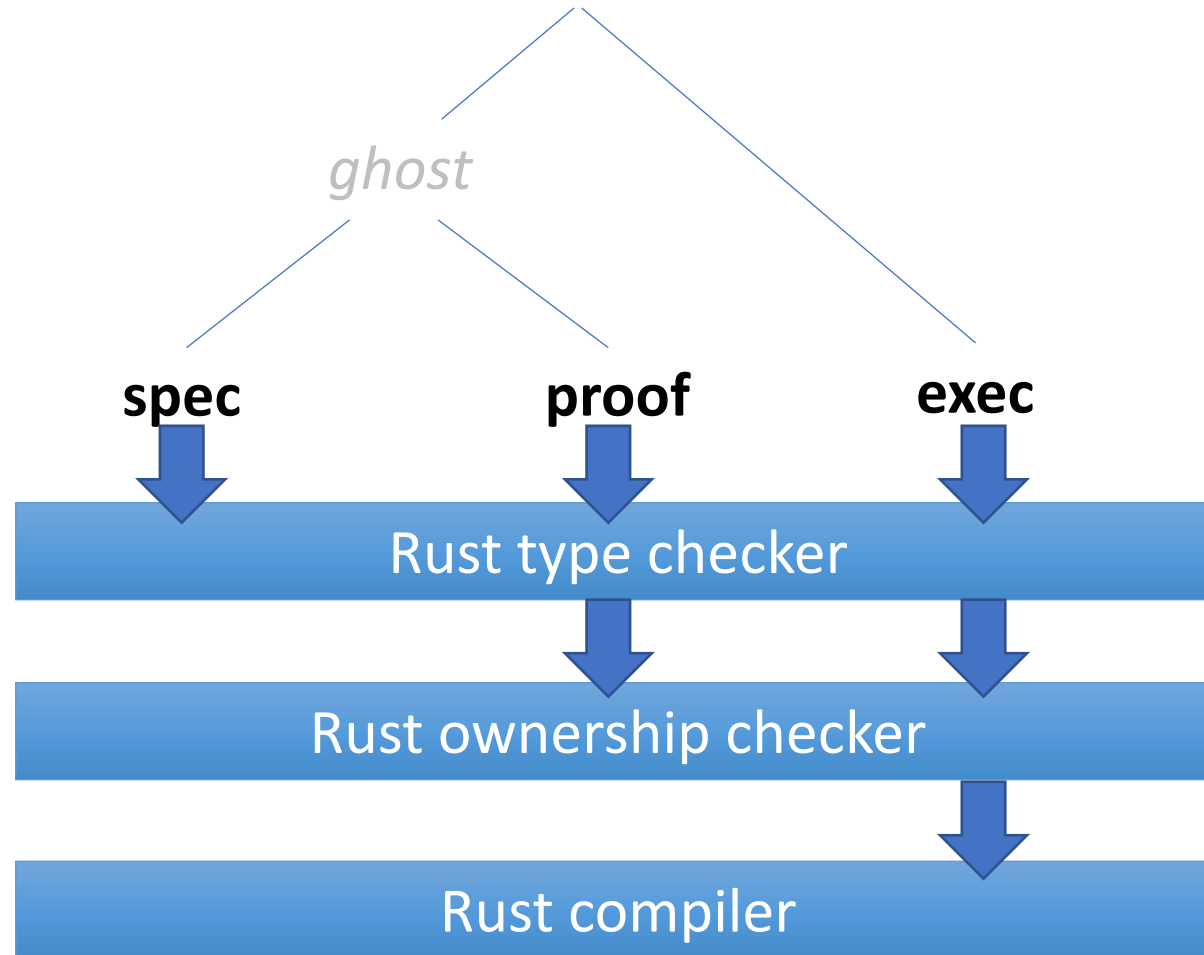
Rust raw pointer type “`*mut t`”
(like “`&mut t`”, but without
the safeguards)

Actual Verus code for Rust unsafe raw pointers (using `vstd::raw_ptr::*`)

```
fn f(x: *mut u32, Tracked(px): Tracked<PointsTo<u32>>,
    y: *mut u32, Tracked(py): Tracked<PointsTo<u32>>)
  requires
    px.value() == 100, px.ptr() == x, px.is_init(),
    py.value() == 200, py.ptr() == y, py.is_init(),
{
  let tracked mut px = px;
  let n = ptr_mut_read(x, Tracked(&mut px));
  ptr_mut_write(x, Tracked(&mut px), n + 1);
  assert(px.value() == 101);
}
```

tracked parameters and arguments in exec code must be wrapped in **Tracked(...)**
(ugly, but it’s a way to cram proofs into Rust’s existing type system)
Tracked is a “zero-size” type and is thus erased during compilation

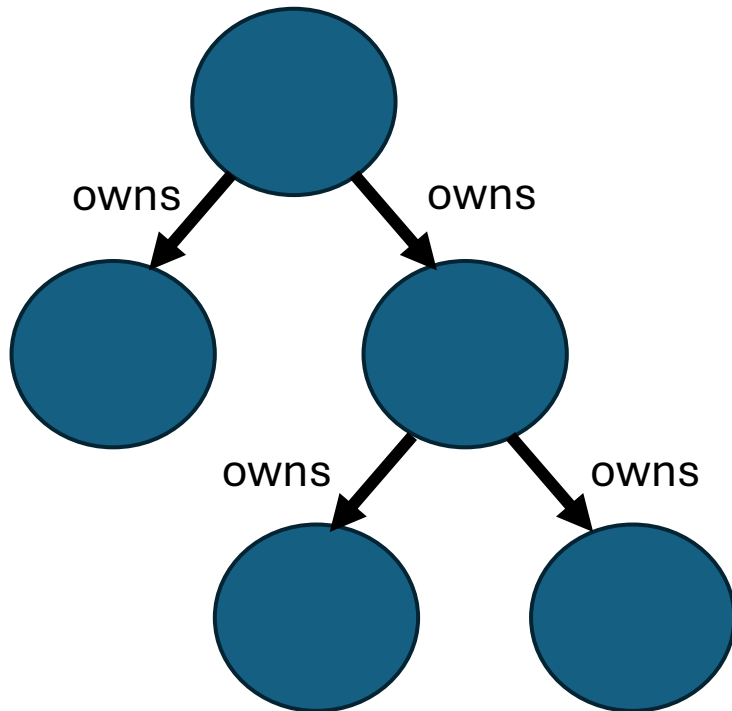
Ownership checking in proof and exec code



Verus can verify safety of “unsafe” Rust code

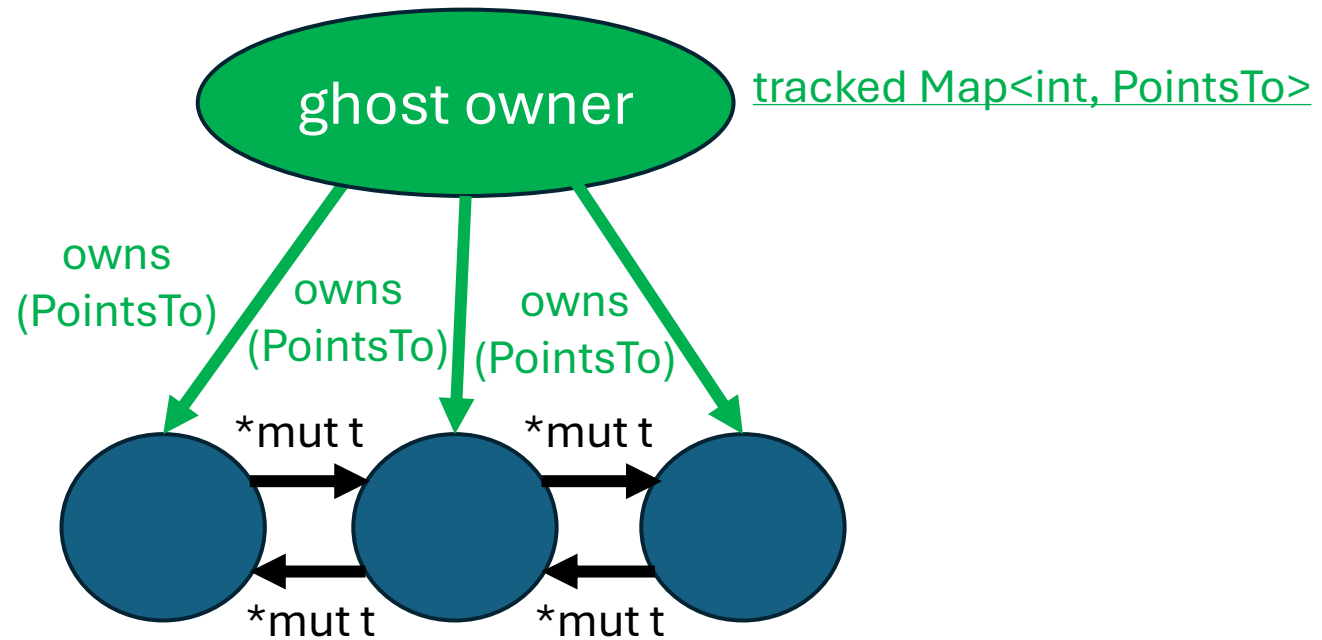
Trees:

Parent node "owns" child nodes;
Rust can check type safety



Doubly linked lists:

next and prev don't own each other,
so ordinary Rust can't check type safety.



Lab exercise: raw pointer ring buffer

```
// Fixed-capacity FIFO queue, pushing at the tail and popping from the head.  
// Each element must be 4 bytes in size.  
struct RingBuffer5<T> {  
    head: usize,  
    tail: usize,  
    // pointer to 0th element of data  
    data_ptr: *mut T,  
    // free and alloc map index 0, 1, 2, 3, 4 to PointsToRaw or PointsTo<T>  
    free: Tracked<Map<int, PointsToRaw>>,  
    alloc: Tracked<Map<int, PointsTo<T>>>,  
}
```

Fractional permissions in separation logic

Split mutable points-to into multiple “fractional” immutable points-to

$$x \mapsto v \iff (x \mapsto^{0.5} v)^* (x \mapsto^{0.5} v) \iff (x \mapsto^{0.5} v)^* ((x \mapsto^{0.1} v)^* (x \mapsto^{0.4} v))$$

Each fractional permission has a real-number fraction between 0 and 1
Together, the fractions add up to 1.

Fractional permissions: Verus example

```
tracked struct FracGhost<T> { ... }
impl<T> FracGhost<T> {
  spec fn id(self) -> Loc;
  spec fn view(self) -> T;
  spec fn frac(self) -> real;

  proof fn new(v: T) -> (tracked r: FracGhost<T>)
    ensures r.frac() == 1.0real, r@ == v;

  proof fn split(tracked &mut self) -> (tracked r: Self)
    ensures
      final(self).frac() == old(self).frac() / 2.0real,
      r.frac() == old(self).frac() / 2.0real,
      ...;

  proof fn combine(tracked &mut self, tracked x: Self)
    requires old(self).id() == x.id(),
    ensures ..., final(self).frac() == old(self).frac() + x.frac(),
    ...
}
```

```
proof fn test() {
  let tracked mut x = FracGhost::new(100int);

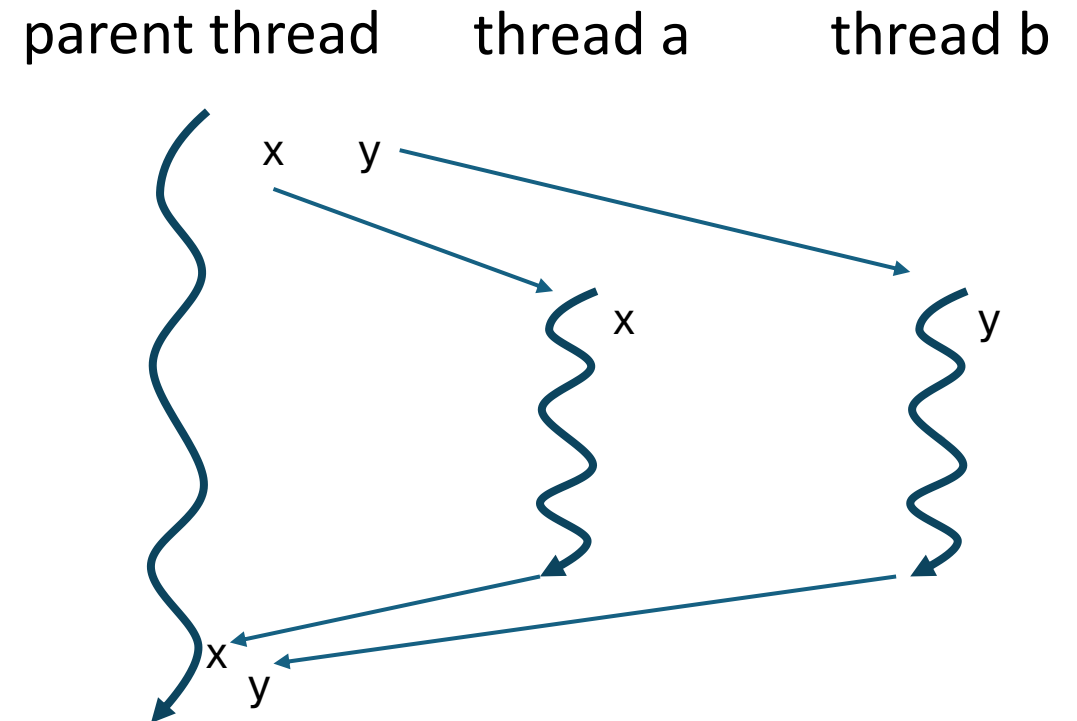
  let tracked y = x.split();
  assert(x.frac() == 0.5real);
  assert(y.frac() == 0.5real);
  assert(x@ == 100);
  assert(y@ == 100);

  x.combine(y);
  assert(x.frac() == 1.0real);
  x.update(200);
  assert(x@ == 200);
}
```

Concurrency

Concurrency via ownership/separation

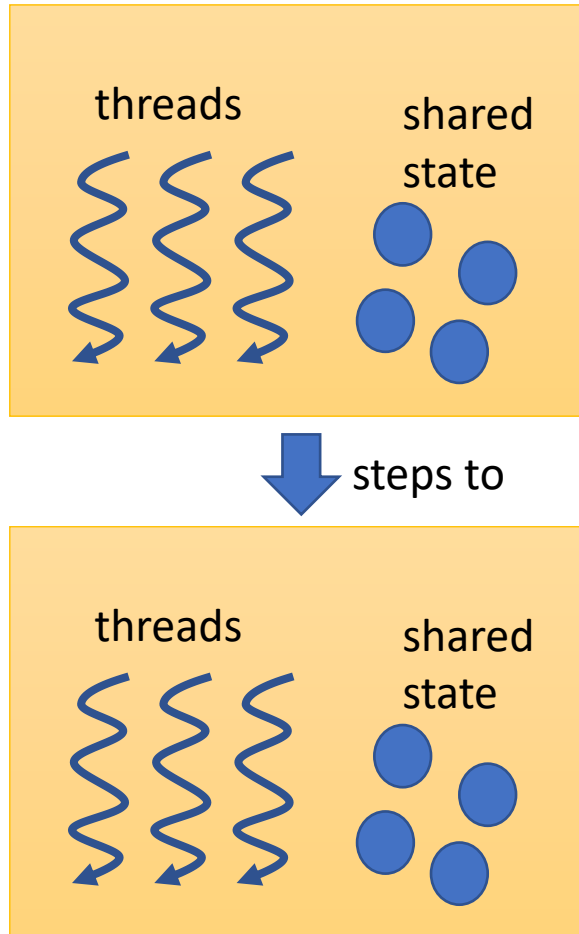
```
fn f(x: Vec<u8>, y: Vec<u8>) {  
    let thread_a = std::thread::spawn(|| {  
        let mut x = x;  
        x.push(10);  
        x  
    });  
  
    let thread_b = std::thread::spawn(|| {  
        let mut y = y;  
        y.push(20);  
        let lx = x.len(); // error: x was moved into thread a  
        y  
    });  
  
    let lx = x.len(); // error: x was moved into thread a  
    if let (Ok(x), Ok(y)) = (thread_a.join(), thread_b.join()) {  
        let lx = x.len();  
        let ly = y.len();  
    }  
}
```



*Concurrency is relatively easy when state is separate
But what if state is shared between threads?*

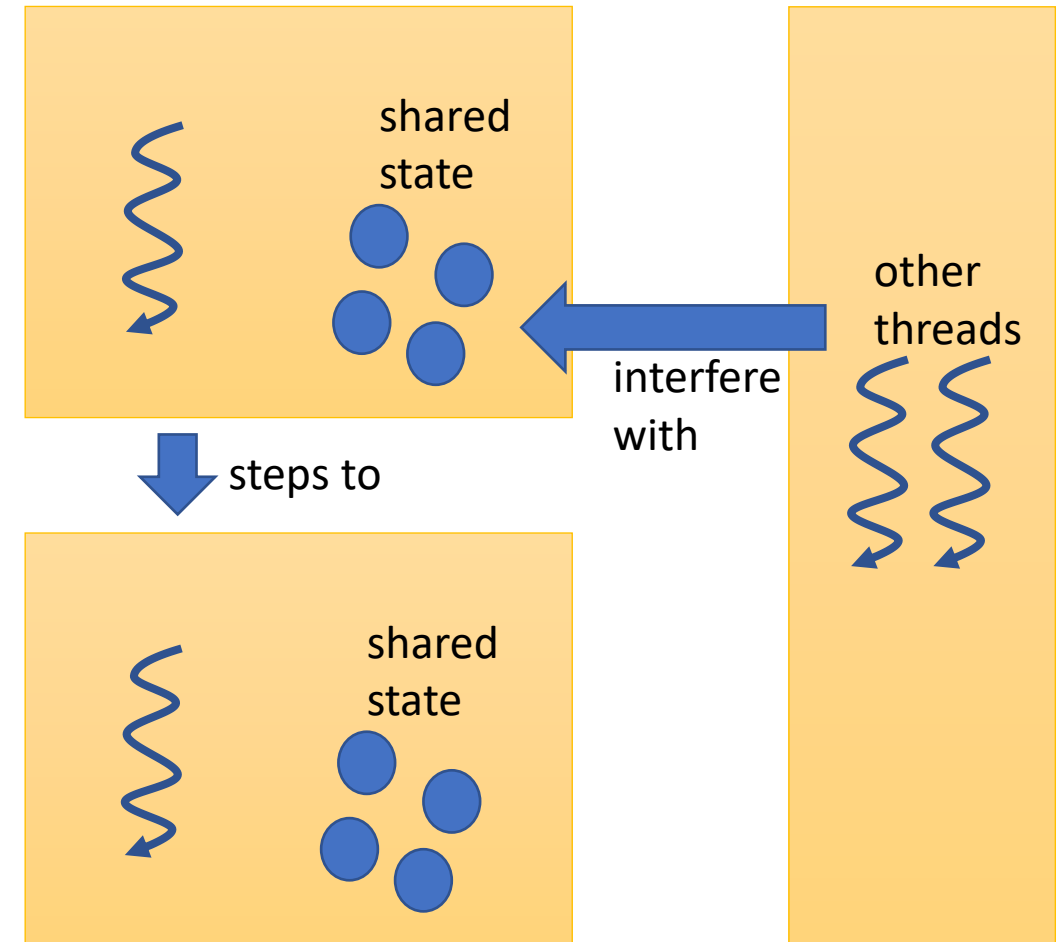
Concurrency with shared state

Perspective #1: whole system



Unwieldy for verifying large systems, though

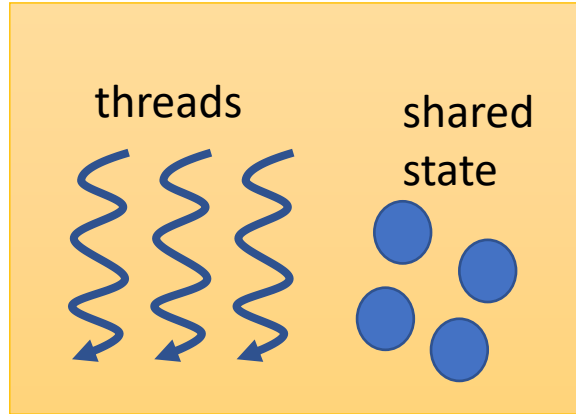
Perspective #2: one thread's view



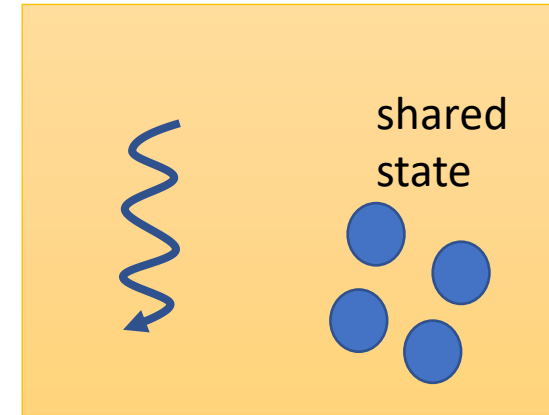
Example: Rely-Guarantee

Concurrency: global protocol, local actions

Perspective #1: whole system



Perspective #2: one thread's view



Describe an abstract protocol
as a whole system.

*Use invariants and/or state machines
to describe global protocol*

Describe each thread's implementation
of the protocol locally.

*Use ghost permissions, ownership
to perform local protocol steps*

Example: concurrer

initial state: counter == 0

thread a



counter.fetch_add

final state: counter == 2

// Unverified Rust code

```
fn unverified_test() {  
    let ptr = AtomicU32::new(0);  
    let ptr = Arc::new(ptr);  
    let ptr_a = ptr.clone();  
    let ptr_b = ptr.clone();  
  
    let thread_a = std::thread::spawn(|| {  
        let ptr = ptr_a;  
        ptr.fetch_add(1, Ordering::SeqCst);  
    });  
  
    let thread_b = std::thread::spawn(|| {  
        let ptr = ptr_b;  
        ptr.fetch_add(1, Ordering::SeqCst);  
    });  
  
    if let (Ok(()), Ok(())) = (thread_a.join(), thread_b.join()) {  
        let n = ptr.load(Ordering::SeqCst);  
        assert!(n == 2);  
    }  
}
```

Example: concurrent count-to-2

initial state: counter == 0

thread a



`counter.fetch_add(1)`

thread b



`counter.fetch_add(1)`

final state: counter == 2

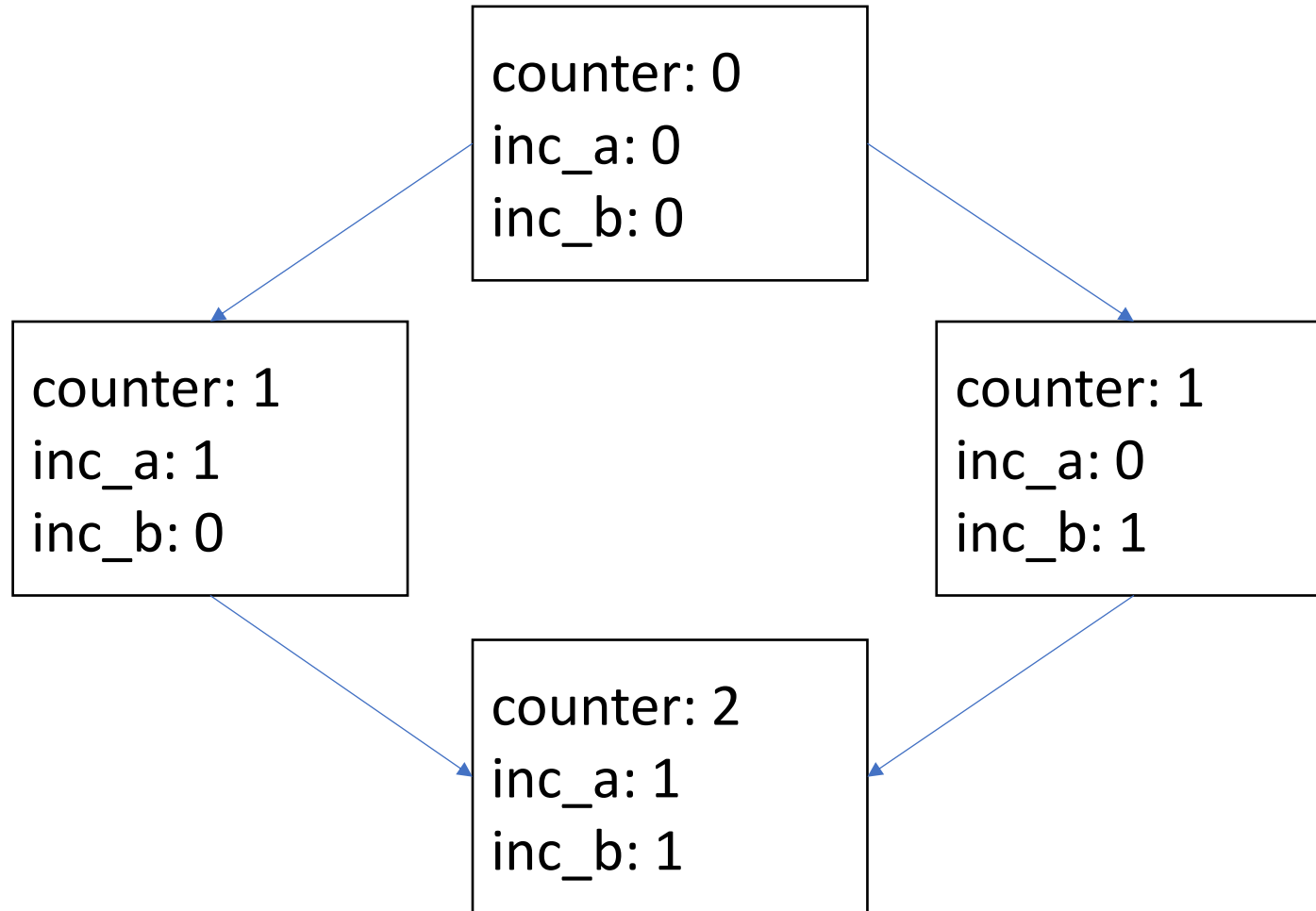
State machine for count-to-2

State type

```
counter: int  
inc_a: int  
inc_b: int
```

invariant:

$\text{counter} = \text{inc_a} + \text{inc_b}$



State machine for count-to-2

State type

```
counter: int  
inc_a: int  
inc_b: int
```

Globally, we have a protocol (a state machine) with an invariant
Locally:

- thread_a is in charge of inc_a
- thread_b is in charge of inc_b

invariant:

$\text{counter} = \text{inc_a} + \text{inc_b}$

Verus supports two different approaches to express global/local cooperation:

- Individual separation-logic-style permissions (a la Iris)
- A dedicated macro-based state machine language

State machine for count-to-2

State type

```
counter: int  
inc_a: int  
inc_b: int
```

Globally, we have a protocol (a state machine) with an invariant
Locally:

- thread_a is in charge of inc_a
- thread_b is in charge of inc_b

invariant:

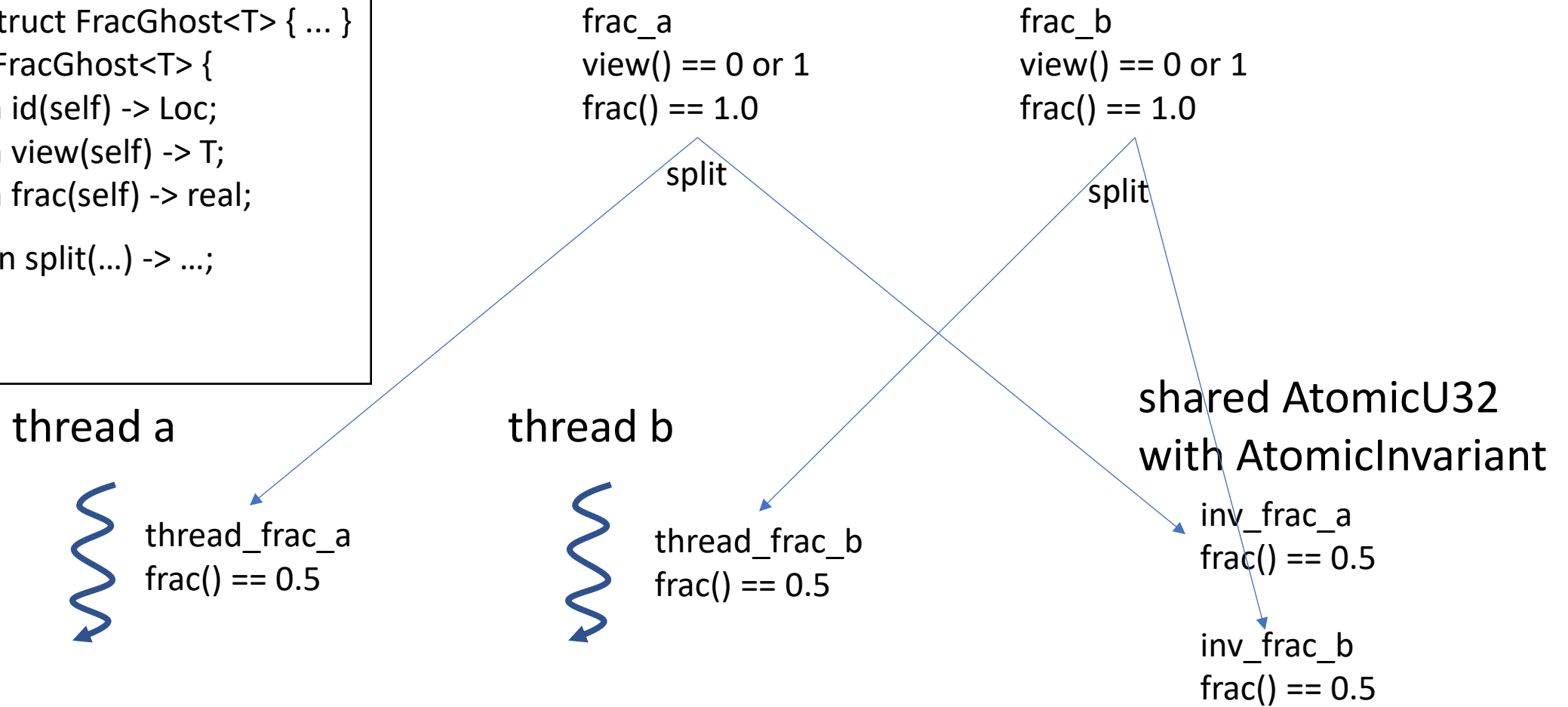
$\text{counter} = \text{inc_a} + \text{inc_b}$

Verus supports two different approaches to express global/local cooperation:

- **Individual separation-logic-style permissions (a la Iris)**
- A dedicated macro-based state machine language

Individual separation-logic-style permissions

```
tracked struct FracGhost<T> { ... }  
impl<T> FracGhost<T> {  
  spec fn id(self) -> Loc;  
  spec fn view(self) -> T;  
  spec fn frac(self) -> real;  
  proof fn split(...) -> ...;  
  ...  
}
```



Split the fractions between the thread and the shared state to coordinate updates
(make sure each thread's view() agrees with the shared states's view())

Thread b performs fetch_add

thread a



thread_frac_a
frac() == 0.5
view() == 0

thread b



thread_frac_b
frac() == 0.5
view() == 0

shared AtomicU32
with AtomicInvariant

inv_frac_a
frac() == 0.5

inv_frac_b
frac() == 0.5
view() == 0

Thread b performs fetch_add

thread a



thread_frac_a
frac() == 0.5
view() == 0

thread b



shared AtomicU32
with AtomicInvariant

inv_frac_a
frac() == 0.5

inv_frac_b
frac() == 1.0
view() == 0

combine

Thread b performs fetch_add

thread a



thread_frac_a
frac() == 0.5
view() == 0

thread b



shared AtomicU32
with AtomicInvariant

inv_frac_a
frac() == 0.5

inv_frac_b
frac() == 1.0
view() == 1

update view()

Thread b performs fetch_add

thread a



thread_frac_a
frac() == 0.5
view() == 0

thread b



shared AtomicU32
with AtomicInvariant

inv_frac_a
frac() == 0.5

thread_frac_ibv_frac_b
frac() == 0.5 frac() == 0.5
view() == 1 view() == 1

split

Parent thread joins thread a, thread b

thread a



thread_frac_a
frac() == 0.5
view() == 1

thread b



thread_frac_b
frac() == 0.5
view() == 1

parent thread

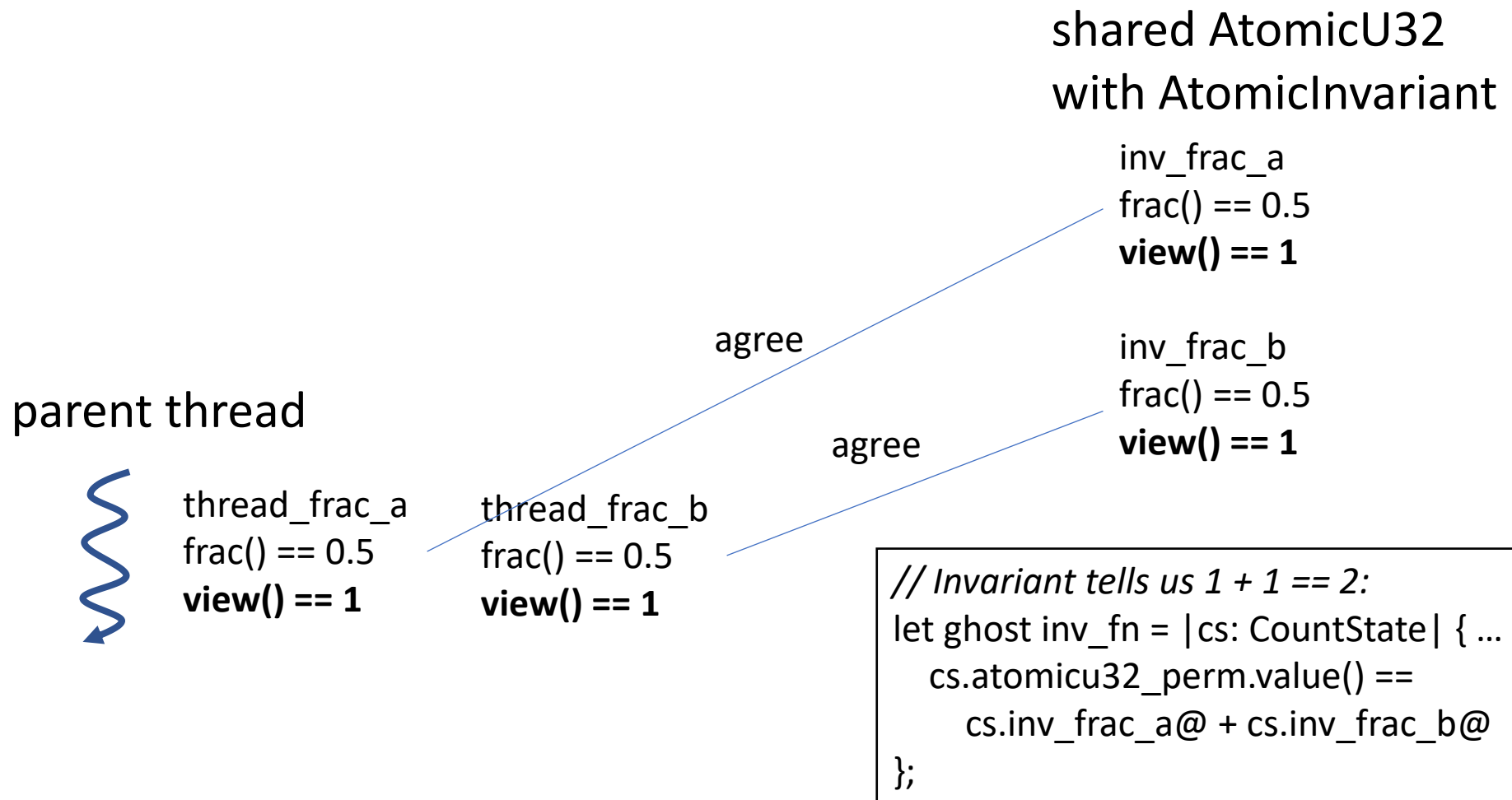


shared AtomicU32
with AtomicInvariant

inv_frac_a
frac() == 0.5
view() == ?

inv_frac_b
frac() == 0.5
view() == ?

Parent thread joins thread a, thread b



State machine for count-to-2

State type

```
counter: int  
inc_a: int  
inc_b: int
```

Globally, we have a protocol (a state machine) with an invariant
Locally:

- thread_a is in charge of inc_a
- thread_b is in charge of inc_b

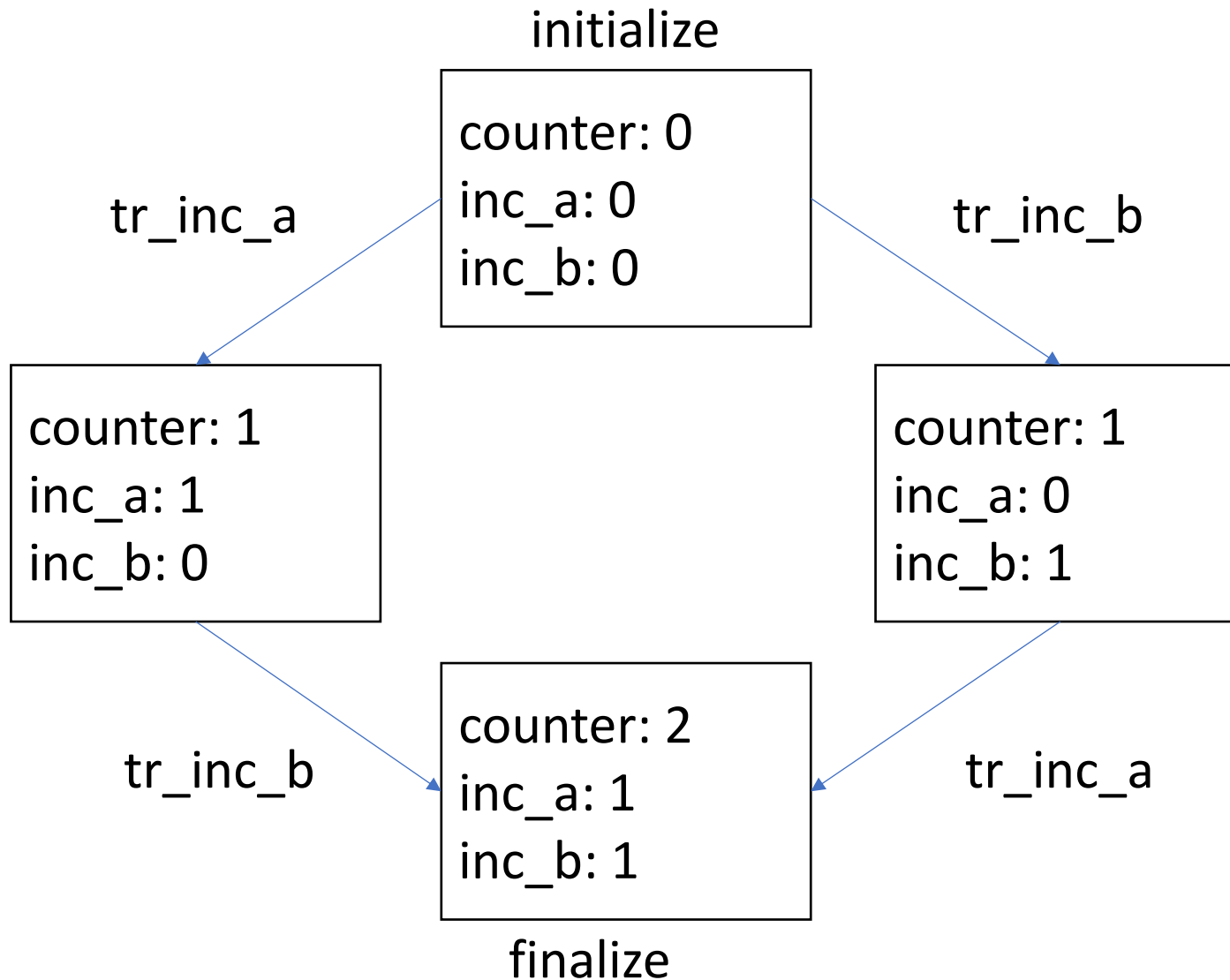
invariant:

$\text{counter} = \text{inc_a} + \text{inc_b}$

Verus supports two different approaches to express global/local cooperation:

- Individual separation-logic-style permissions (a la Iris)
- **A dedicated macro-based state machine language**

State machine for count-to-2



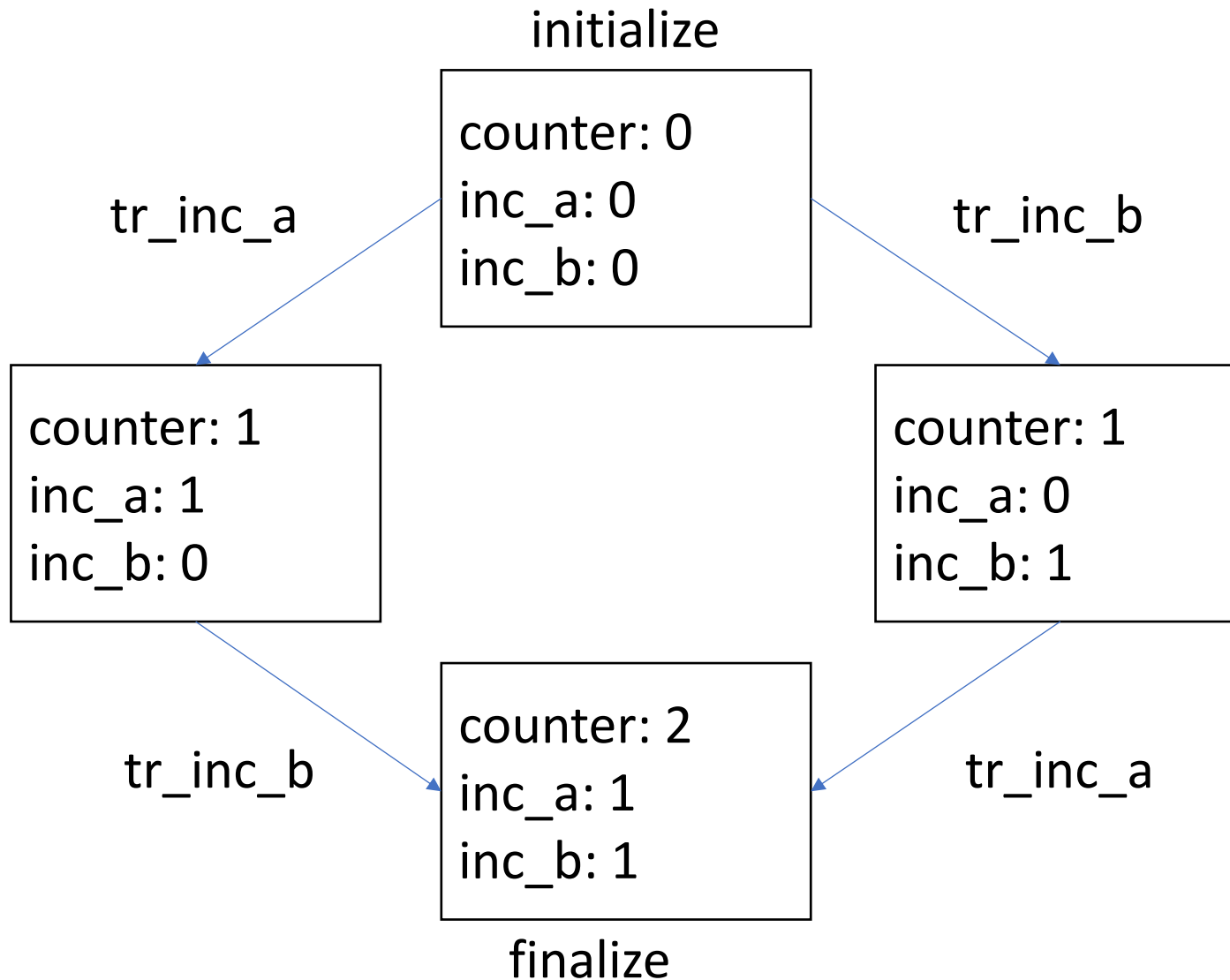
State type

```
counter: int
inc_a: int
inc_b: int
```

invariant:

$\text{counter} = \text{inc_a} + \text{inc_b}$

State machine for count-to-2



```
// Slightly simplified to fit on slide
tokenized_state_machine!( Counter {
  fields {
    counter: int, inc_a: int, inc_b: int,
  }
  #[invariant] fn inv(&self) -> bool {
    ...
    self.counter == self.inc_a + self.inc_b
  }
  init!{ initialize() {
    init counter = 0; init inc_a = 0; init inc_b = 0;
  }}
  transition!{ tr_inc_a() {
    require(pre.inc_a == 0);
    update counter = pre.counter + 1;
    update inc_a = 1;
  }}
  ...
}
```

State machine for count-to-2

State machine generates
tracked permissions for threads

```
...
let tracked (
    Tracked(instance),
    Tracked(counter),
    Tracked(inc_a),
    Tracked(inc_b),
) = Counter::Instance::initialize();
...
let thread_a = vstd::thread::spawn(|| {
    ... inc_a ...
});
let thread_b = vstd::thread::spawn(|| {
    ... inc_b ...
});
...
```

```
// Slightly simplified to fit on slide
tokenized_state_machine!( Counter {
    fields {
        counter: int, inc_a: int, inc_b: int,
    }
    #[invariant] fn inv(&self) -> bool {
        ...
        self.counter == self.inc_a + self.inc_b
    }
    init!{ initialize() {
        init counter = 0; init inc_a = 0; init inc_b = 0;
    }}
    transition!{ tr_inc_a() {
        require(pre.inc_a == 0);
        update counter = pre.counter + 1;
        update inc_a = 1;
    }}
    ...}
}
```

Any questions?